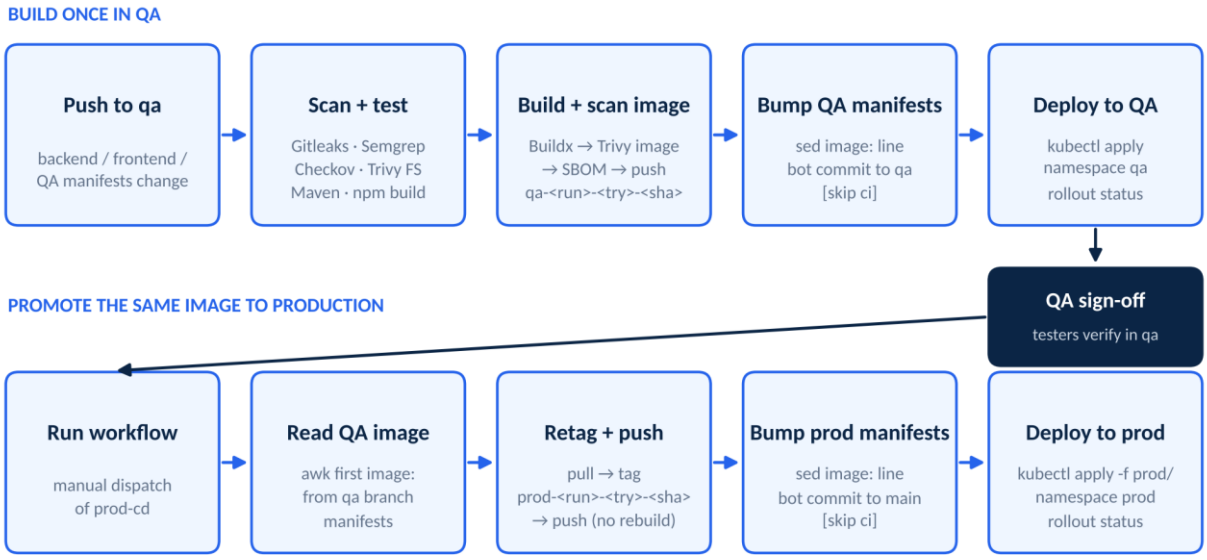
DEVOPS SHACK

Enterprise DevSecOps CI/CD on GitHub Actions and Amazon EKS

The QA pipeline and the production promotion pipeline, explained step by step and line by line

`qa-cicd.yaml` · `prod-cd.yaml`

Gitleaks Semgrep Checkov Trivy SonarQube SBOM OIDC ECR EKS



Aditya Jaiswal · DevOps Shack

devopsshack.com · October 2026

BEFORE WE START

How to Read This Guide

This guide walks through the two GitHub Actions workflows that take our application from a commit on the `qa` branch all the way to a production rollout on Amazon EKS. Every job, every step and every line is explained — what it does, why it is there, and what would break if it were missing.

The two files we cover are the ones that sit in `.github/workflows/` of the project:

- `qa-cicd.yaml` — **"Enterprise DevSecOps QA CI/CD".** Runs automatically on every relevant push to `qa`. It scans the code, tests it, builds the backend and frontend container images, scans those images, pushes them to Amazon ECR, records the new image tags in the QA Kubernetes manifests and deploys them to the `qa` namespace.
- `prod-cd.yaml` — **"Enterprise Production CD Pipeline".** Runs only when someone presses **Run workflow**. It takes the exact images that were tested in QA, gives them production tags without rebuilding anything, records those tags in the production manifests on `main`, and deploys them to the `prod` namespace.

The application itself is a classic two-image setup: a **Java 21 / Maven backend** and a **Node.js 22 frontend**, each with its own Dockerfile, each pushed to its own ECR repository (`projectops-backend` and `projectops-frontend`). Both environments live on the same EKS cluster, `my-cluster`, in `ap-south-1` (Mumbai), separated by namespace.

How each section is laid out

Every job gets its own section. We first say in plain words what the job is for and where it sits in the pipeline. Then we show the YAML exactly as it appears in the file — the grey numbers on the left are the **real line numbers** in the workflow, so you can keep the file open side by side. Under each code block, the line notes walk through it top to bottom.

Along the way you will see four kinds of boxes:

> WHY WE DO IT THIS WAY
> The reasoning behind a design choice — the part interviewers usually ask about.

> WATCH OUT
> A behaviour that surprises people, or something that will bite you when you change the pipeline.

> IN PRACTICE
> How the step shows up when you actually run it — logs, artifacts, what to check.

What is inside

1. **The big picture** — the build-once, promote-later idea that both files are built around.
2. **What must exist first** — the repository layout, GitHub secrets, the AWS OIDC roles, ECR and EKS access.
3. **The QA pipeline, line by line** — the workflow header and all ten jobs.
4. **The production pipeline, line by line** — the header and all three jobs.
5. **A release, end to end** — what one real QA run and production promotion look like together.
6. **Hardening notes** — what we tighten when this pipeline moves from “report” to “enforce”.
7. **Quick reference** — secrets, tags, artifacts and commands on one page.

CHAPTER 1

The Big Picture: Build Once, Promote Later

Both workflows are built around a single rule: the image that runs in production is the exact image that was tested in QA. We never rebuild for production.

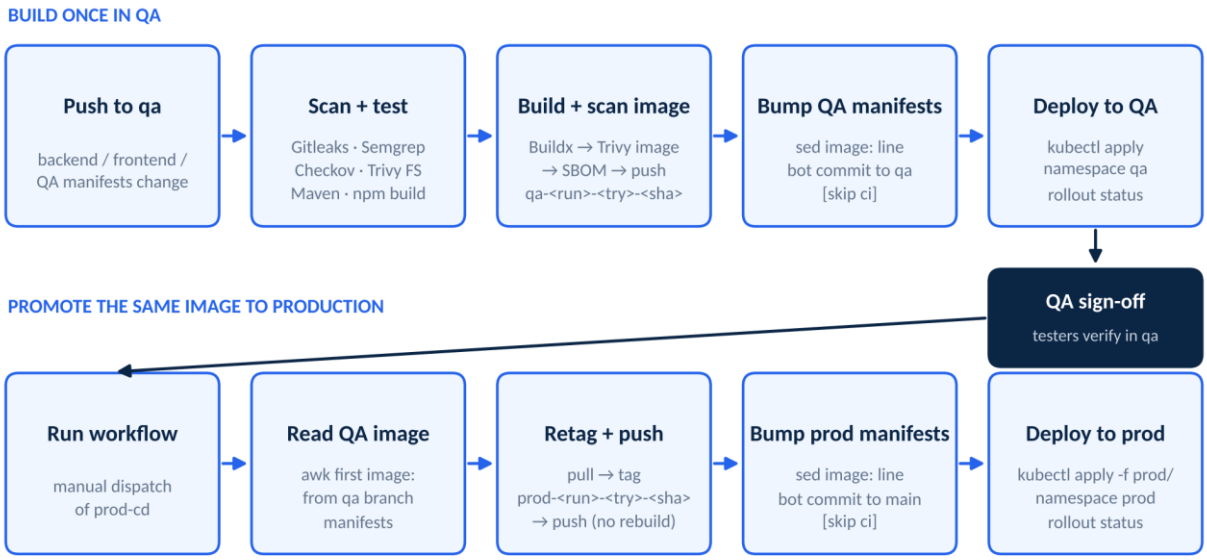


Figure 1 — The QA pipeline builds and deploys; after QA sign-off, the production pipeline promotes the same image.

If we rebuilt the image for production, even from the same commit, we could end up with something different from what QA tested: a base image that moved, a dependency resolved to a newer patch, a build cache that behaved differently. The production artifact would be untested in the strict sense. So the QA pipeline is the only place an image is ever built. The production pipeline only **renames** it — it pulls the QA-tested image, adds a production tag to the very same image, and pushes that tag.

That gives us a clean split of responsibilities:

- **The `qa` branch is the source of truth for "what is in QA".** The QA pipeline writes the new image tags into `k8-manifests/qa/backend.yaml` and `k8-manifests/qa/frontend.yaml` and commits them back to `qa`. Anyone can open those files and see exactly which image QA is running.
- **The `main` branch is the source of truth for "what is in production".** The production pipeline writes the promoted tags into `k8-manifests/prod/` and commits them to `main`.
- **The production pipeline reads the QA manifest to decide what to promote.** It does not guess and it does not take a tag as input — it reads the `image:` line that QA is currently running. Whatever passed QA is what gets promoted.
- **Two separate IAM roles.** QA jobs assume a QA role (`AWS_QA_ROLE_TO_ASSUME`) and production jobs assume a production role (`AWS_PROD_ROLE_TO_ASSUME`). The production role is deliberately scoped to the `prod` namespace only, so a mistake in QA can never touch production.

PUSH-BASED DEPLOYS WITH A GIT RECORD

Both pipelines deploy with `kubectl apply` straight from GitHub Actions — this is a push-based deployment, not a GitOps controller like Argo CD pulling from Git. But because every deployment first commits the new image tag to Git, the repository still holds a full, reviewable history of what was deployed and when. If we later add Argo CD, it can watch those same manifest folders and the pipelines barely change.

A quick word on DevSecOps

The QA pipeline runs seven different security and quality checks before an image is pushed. Each one looks at a different layer, which is why we do not stop at one scanner:

Tool	What it looks at	What it catches
Gitleaks	Git commits	API keys, passwords and tokens committed to the repository
Semgrep	Application source code	Insecure code patterns (SAST) — injection, weak crypto, unsafe APIs
Checkov	Kubernetes YAML and Dockerfiles	Misconfiguration — running as root, no resource limits, `latest` tags
Trivy FS	Dependency files in the source tree	Known CVEs in Maven and npm dependencies
SonarQube	Source code and tests	Bugs, code smells, duplication, coverage — and the quality gate
Trivy image	The built container image	CVEs in the OS packages and libraries inside the final image
Syft (SBOM)	The built container image	Produces a full inventory of what is in the image, for audit and later re-scanning

EVERY SCANNER HERE RUNS IN REPORT MODE

Look closely and you will see `--exit-code 0`, `|| true` or `continue-on-error: true` on every scanner. That means the scanners **record** findings and upload reports, but they never stop the pipeline. This is a common and sensible starting point — you get visibility without blocking the team on day one — but it is not enforcement. Chapter 6 covers how we switch each one to blocking mode.

CHAPTER 2

What Must Exist Before Either Pipeline Runs

The YAML assumes a lot about the world around it. If any of these pieces is missing, the pipeline fails at a step that often looks unrelated to the real cause. It is worth getting this list right first.

1. The repository layout

The workflows refer to these paths directly, so the repository has to look like this:

```
.
├── .github/workflows/
│   ├── qa-cicd.yaml
│   └── prod-cd.yaml
├── backend/                # Java 21, Maven
│   ├── pom.xml
│   ├── Dockerfile
│   └── src/main/java, src/test/java
├── frontend/               # Node.js 22
│   ├── package.json
│   ├── package-lock.json   # required by npm ci and the npm cache
│   ├── Dockerfile
│   └── src/
└── k8-manifests/
    ├── qa/    backend.yaml  frontend.yaml  ingress.yaml
    └── prod/  backend.yaml  frontend.yaml  ingress.yaml
```

Two branches matter: `qa`, which triggers the QA pipeline and holds the QA manifests, and `main`, which holds the production manifests. The production pipeline checks out both.

> **THE MANIFESTS MUST DECLARE THEIR OWN NAMESPACE**
>
> Neither pipeline passes `-n` to `kubectl apply`. The kubeconfig created by `aws eks update-kubeconfig` points at the `default` namespace, so every Deployment, Service and Ingress in `k8-manifests/qa/` must contain `namespace: qa` in its metadata, and every file in `k8-manifests/prod/` must contain `namespace: prod`. The rollout checks do pass `-n`, so a missing namespace shows up as a confusing "deployment not found" at the rollout step.
>
> The rollout steps also expect the Deployments to be named exactly `backend` and `frontend`.

2. GitHub secrets and variables

These live under **Settings → Secrets and variables → Actions** in the repository:

Name	Type	Used by	Holds
`AWS_QA_ROLE_TO_ASSUME`	Secret	QA: docker, deploy-to-k8s	ARN of the QA IAM role
`AWS_PROD_ROLE_TO_ASSUME`	Secret	Prod: promote, deploy	ARN of the production IAM role

Name	Type	Used by	Holds
`SONAR_TOKEN`	Secret	QA: sonarqube	SonarQube user/project token
`SONAR_HOST_URL`	Secret	QA: sonarqube	e.g. `https://sonar.example.com`
`SONAR_PROJECT_KEY`	Variable (optional)	QA: sonarqube	Project key; falls back to the repo name
`GITHUB_TOKEN`	Automatic	Both: manifest commits	Created by GitHub for every run

Notice there is **no AWS access key anywhere**. Both pipelines authenticate with OpenID Connect (OIDC), which is the next item.

3. The AWS side: OIDC and two IAM roles

With OIDC, GitHub issues each job a short-lived, signed token that says "I am a workflow run in repository X, on branch Y". AWS checks the signature, checks the claims against the role's trust policy, and hands back temporary credentials that expire within the hour. Nothing long-lived is stored in GitHub.

To make that work, the AWS account needs:

- **An IAM OIDC identity provider** for `token.actions.githubusercontent.com` with audience `sts.amazonaws.com`. This is created once per AWS account.
- **A QA role** whose trust policy only accepts tokens from this repository's `qa` branch, with permissions to push and pull ECR images and to call `eks:DescribeCluster`.
- **A production role** whose trust policy only accepts tokens from `main`, with ECR pull/push on the same two repositories and `eks:DescribeCluster`.

A trust policy for the QA role looks like this:

```
{
  "Effect": "Allow",
  "Principal": {
    "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/token.actions.githubusercontent.com"
  },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:<OWNER>/<REPO>:ref:refs/heads/qa"
    }
  }
}
```

The production role is identical except the `sub` condition ends in `refs/heads/main`. Because the production workflow is started manually, the `sub` claim reflects whichever branch the person picks in the **Run workflow** dropdown — so locking the production role to `main` also means nobody can run a production deploy from a feature branch.

4. ECR repositories

Two private repositories in `ap-south-1`: `projectops-backend` and `projectops-frontend`. QA and production tags live side by side in the same repositories — a QA image is `projectops-backend:qa-42-1-a1b2c3d`, and after promotion the same image is also `projectops-backend:prod-17-1-9f8e7d6`.

5. Kubernetes access on EKS

IAM decides who can *talk to* the cluster; Kubernetes RBAC decides what they can *do* inside it. Each IAM role is mapped to Kubernetes permissions with an **EKS access entry** (or the older `aws-auth` ConfigMap):

- The **QA role** needs to create and update Deployments, Services and Ingresses in the `qa` namespace, and read Pods (for the final `kubectl get`) and watch rollouts.
- The **production role** gets the same in `prod`, plus ConfigMaps and Secrets — the production deploy explicitly checks for all five. It gets **nothing cluster-wide**, which is why the production YAML warns against `kubectl get nodes`.

Both namespaces have to exist before the first run; the pipelines do not create them.

6. SonarQube

A SonarQube server reachable from GitHub-hosted runners (so not on a private network unless you use self-hosted runners), a project for this repository, and a token with permission to analyse it.

CHAPTER 3 · QA-CICD.YAML

The QA Pipeline, Line by Line

This is the workhorse. Every push to `qa` that touches the application or its QA manifests runs ten jobs: six checks in parallel, an image build, a SonarQube analysis, a manifest update and a deployment.

The workflow header `lines 1–29`

Everything before `jobs:` applies to the whole workflow: its name, what starts it, the default permissions, how overlapping runs are handled, and a set of shared variables.

```
1   name: Enterprise DevSecOps QA CI/CD
2
3   on:
4     push:
5       branches:
6         - qa
7       paths:
8         - 'backend/**'
9         - 'frontend/**'
10        - 'k8-manifests/qa/backend.yaml'
11        - 'k8-manifests/qa/frontend.yaml'
12        - 'k8-manifests/qa/ingress.yaml'
13        - '.github/workflows/qa-cicd.yaml'
14
15    workflow_dispatch:
```

Line 1 — `name` is what appears in the **Actions** tab and in the status checks on commits. It has no effect on behaviour.

Line 3 — `on:` lists the events that start the workflow. There are two: `push` and `workflow_dispatch`.

Lines 4–6 — `push` with `branches: [qa]` — only pushes to the `qa` branch count. A push to `main` or a feature branch does nothing here. Merging a pull request into `qa` is also a push to `qa`, so merged PRs trigger it too.

Lines 7–13 — `paths:` narrows it further: the push must change at least one file matching these patterns. `backend/**` and `frontend/**` mean “anything under those folders”, at any depth. The three QA manifests are listed individually, so editing the ingress or a Deployment spec redeploys QA. The workflow file itself is listed so that changing the pipeline immediately tests the change. A push that only edits, say, the README is ignored.

Line 15 — `workflow_dispatch:` adds a **Run workflow** button in the Actions tab, so we can re-run QA by hand — for example after rotating a secret — without making a dummy commit. A manual run ignores the `paths` filter.

THE PIPELINE'S OWN COMMIT MATCHES ITS PATHS FILTER

Later, the `update-qa-manifests` job commits changes to `k8-manifests/qa/backend.yaml` and `frontend.yaml` — which are in the `paths` list. It does not cause an endless loop for two independent reasons: pushes made with the built-in `GITHUB_TOKEN` never start new workflow runs, and the commit message contains `[skip ci]`, which GitHub also honours. Either one alone would be enough; together they are belt and braces.

```
17  permissions:
18    contents: read
19
20  concurrency:
21    group: qa-${{ github.ref }}
22    cancel-in-progress: true
23
24  env:
25    AWS_REGION: ap-south-1
26    EKS_CLUSTER_NAME: my-cluster
27    K8S_NAMESPACE: qa
28    BACKEND_REPO: projectops-backend
29    FRONTEND_REPO: projectops-frontend
```

Lines 17–18 `permissions: contents: read` sets the default permission of the automatic `GITHUB_TOKEN` for every job to read-only. This is least privilege: the scanning jobs only need to read the code. Jobs that need more — pushing a commit, requesting an OIDC token — raise their own permissions explicitly, as we will see.

Lines 20–22 `concurrency` puts every run into a named group. The group name is `qa-` followed by the branch ref, so all runs on `qa` share the group `qa-refs/heads/qa`. With `cancel-in-progress: true`, when a new run starts, any older run still in progress in that group is cancelled. If three commits land in quick succession, only the last one is built and deployed — we do not waste runner time on images that are already outdated, and two runs never race to deploy QA at the same time.

Lines 24–29 `env:` defines variables visible to every step of every job as normal environment variables. `AWS_REGION` and `EKS_CLUSTER_NAME` say where the cluster is, `K8S_NAMESPACE` is the namespace QA deploys into, and the two `_REPO` names are the ECR repository names (without the registry hostname — that is added later, once we have logged in to ECR and know it). Putting them here means changing the region or cluster is a one-line edit.

How the ten jobs fit together

Jobs in GitHub Actions run **in parallel by default**, each on its own fresh virtual machine. The only thing that orders them is `needs:`. Reading the `needs:` lines gives us this graph:

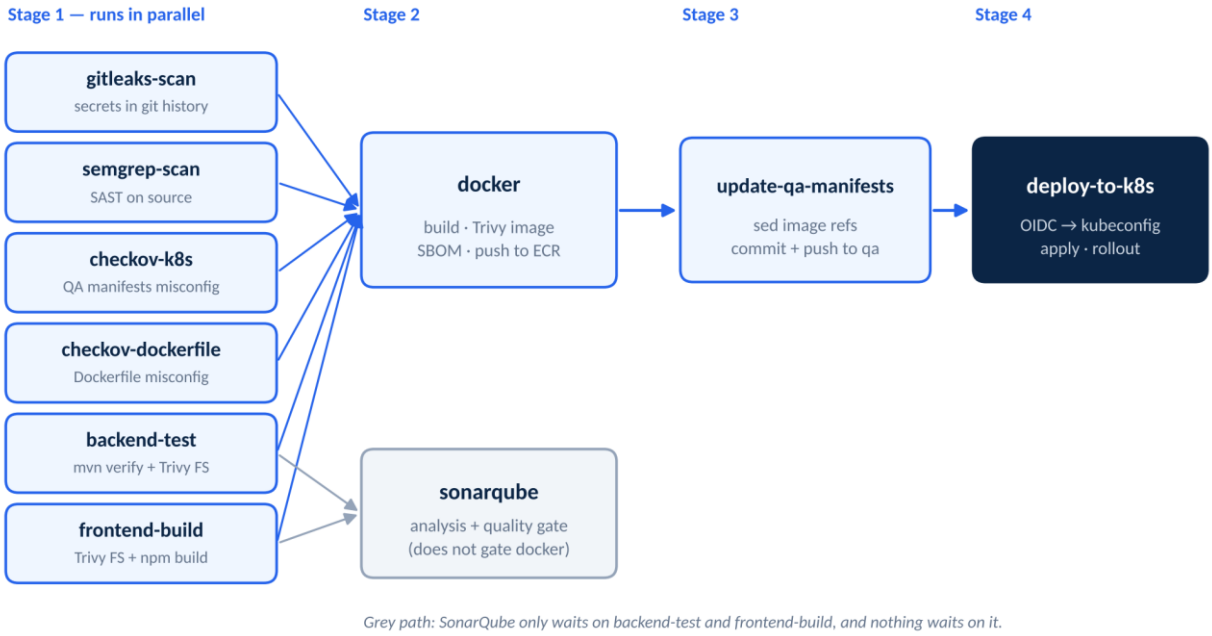


Figure 2 — Job graph of qa-cicd.yaml, derived from its needs: declarations.

- **Stage 1:** six jobs have no `needs:` at all, so they all start at once — `gitleaks-scan`, `semgrep-scan`, `checkov-k8s`, `checkov-dockerfile`, `backend-test` and `frontend-build`.
- **Stage 2:** `docker` waits for all six. `sonarqube` waits only for `backend-test` and `frontend-build`, then runs alongside `docker`.
- **Stage 3 and 4:** `update-qa-manifests` waits for `docker`, and `deploy-to-k8s` waits for `update-qa-manifests`.

Because each job runs on a separate machine, nothing carries over between them automatically — not the checked-out code, not installed tools, not AWS credentials. That is why almost every job starts with its own checkout, and why the deploy job logs in to AWS again even though `docker` already did. Data moves between jobs only through **job outputs** (small strings like image names) and **artifacts** (files like scan reports).

> **SONARQUBE DOES NOT GATE THE RELEASE**
> Nothing lists `sonarqube` in its `needs:`. So `docker`, the manifest update and the deployment can all finish while SonarQube is still analysing, and a failed quality gate cannot stop a QA deployment. If you want the gate to be a gate, add `sonarqube` to the `needs:` of the `docker` job and remove `continue-on-error` from the quality gate step.

Job 1 — gitleaks-scan `lines 33–57`

Gitleaks looks for secrets — AWS keys, GitHub tokens, private keys, passwords in connection strings — that have been committed to the repository. A leaked secret in Git is a real incident even after you delete the file, because it stays in the history.

```
33  gitleaks-scan:
34    name: Gitleaks Scan
35    runs-on: ubuntu-latest
36
37    steps:
38      - name: Checkout source code
39        uses: actions/checkout@v7
```

Line 33 `gitleaks-scan:` is the job ID. Other jobs refer to it by this ID in `needs:`.

Line 34 `name:` is the human-friendly label shown in the run's graph.

Line 35 `runs-on: ubuntu-latest` asks for a fresh GitHub-hosted Ubuntu virtual machine. It comes with Docker, Git, Python, Java, Node, the AWS CLI and more already installed.

Line 37 `steps:` begins the ordered list of things this job does. Steps inside one job run one after another on the same machine and share its filesystem.

Lines 38–39 `actions/checkout` clones the repository into the runner's workspace (`$GITHUB_WORKSPACE`) at the commit that triggered the run. With no options, it fetches only that single commit — a shallow clone with a depth of 1.

```
41  - name: Run Gitleaks scan
42    run: |
43      docker run --rm \
44        -v "${{ github.workspace }}:/repo" \
45        zricethezav/gitleaks:latest detect \
46        --source=/repo \
47        --report-format=json \
48        --report-path=/repo/gitleaks-report.json \
49        --redact \
50        --exit-code 0
```

Lines 41–42 A `run:` step executes shell commands. The `|` starts a multi-line block; the backslashes at the end of each line join them into one long command.

Line 43 `docker run --rm` starts a container and deletes it when it exits. Instead of installing Gitleaks on the runner we use the official image.

Line 44 `-v "${{ github.workspace }}:/repo"` mounts the checked-out code into the container at `/repo`. `${{ github.workspace }}` is an expression GitHub replaces with the workspace path before the shell ever sees it.

Line 45 `zricethezav/gitleaks:latest` is the Gitleaks image, and `detect` is the scan command. `detect` scans the Git history of the repository it is pointed at. (Newer Gitleaks releases also offer `git` and `dir` subcommands; `detect` still works.)

Line 46 `--source=/repo` — scan the mounted repository.

Line 47 `--report-format=json` — write findings as JSON, easy to read by tools and dashboards.

Line 48 `--report-path=/repo/gitleaks-report.json` — the report is written inside the mounted folder, so it lands in the workspace on the runner where the next step can find it.

Line 49 `--redact` — replace the secret values with `REDACTED` in the output. Without this, the report and the job log would contain the very secret we are trying to protect, readable by anyone who can see the run.

Line 50 `--exit-code 0` — Gitleaks normally exits with code 1 when it finds a leak, which fails the step. Forcing 0 means the job always succeeds and the findings are only recorded in the report.

> **A SHALLOW CLONE LIMITS WHAT GITLEAKS SEES**
> Because the checkout on line 39 fetches only the latest commit, Gitleaks only scans that one commit — not the full history. That catches new leaks in the pushed change, which is the main goal, but a secret that arrived in an earlier commit of the same push can be missed. Adding `with: fetch-depth: 0` to the checkout gives Gitleaks the whole history. Pinning the image to a version (instead of `latest`) also keeps results reproducible.

```
52   - name: Upload Gitleaks report
53     uses: actions/upload-artifact@v7
54     with:
55       name: gitleaks-report
56       path: gitleaks-report.json
57       if-no-files-found: warn
```

Line 53 `actions/upload-artifact` saves files from the runner so they survive after the machine is destroyed. They appear in the **Artifacts** section at the bottom of the run summary page.

Line 54 `with:` passes inputs to the action.

Line 55 `name: gitleaks-report` — the name the artifact is listed and downloaded under.

Line 56 `path:` — which file (or files, or folders) to upload.

Line 57 `if-no-files-found: warn` — if the report is missing, log a warning instead of failing. (Gitleaks may skip writing a report when there are no findings, so a missing file is not necessarily a problem.)

Job 2 — semgrep-scan `lines 59–89`

Semgrep is our SAST tool — static application security testing. It reads the source code itself and matches it against rules for insecure patterns: SQL built by string concatenation, disabled TLS verification, hard-coded credentials, unsafe deserialisation and so on.

```
59  semgrep-scan:
60    name: Semgrep SAST Scan
61    runs-on: ubuntu-latest
62
63    steps:
64      - name: Checkout source code
65        uses: actions/checkout@v7
66
67      - name: Setup Python
68        uses: actions/setup-python@v7
69        with:
70          python-version: "3.13"
71
72      - name: Install Semgrep
73        run: pip install semgrep
```

Lines 59–61 Job ID, display name, and a fresh Ubuntu runner, same as before.

Lines 64–65 Check out the code.

Lines 67–70 `actions/setup-python` installs Python 3.13 and puts it on the `PATH`. The version is quoted so YAML treats it as a string, not a number (unquoted, `3.10` would become `3.1`).

Lines 72–73 `pip install semgrep` installs the Semgrep CLI from PyPI.

```
75  - name: Run Semgrep scan
76    run: |
77      semgrep scan \
78        --config auto \
79        ./backend \
80        ./frontend \
81        --json \
82        --output semgrep-report.json || true
```

Line 77 `semgrep scan` runs the scanner.

Line 78 `--config auto` tells Semgrep to pick rule sets automatically based on the languages and frameworks it detects — here, Java and JavaScript/TypeScript rules. The rules are downloaded from the Semgrep registry at run time, so the runner needs internet access.

Lines 79–80 The two folders to scan. Scanning only `backend` and `frontend` keeps the results focused on application code.

Line 81 `--json` — machine-readable output.

Line 82 `--output semgrep-report.json` writes it to a file. `|| true` means “if the command fails, run `true`, which always succeeds” — so the step never fails because of findings.

```
84  - name: Upload Semgrep report
85    uses: actions/upload-artifact@v7
86    with:
87      name: semgrep-report
88      path: semgrep-report.json
89      if-no-files-found: warn
```

Lines 84–89 Upload the report as the `semgrep-report` artifact, exactly like the Gitleaks step.

Job 3 — checkov-k8s `lines 91–118`

Checkov is a policy-as-code scanner for infrastructure. Here it reads our QA Kubernetes manifests and reports misconfigurations: containers running as root, no CPU or memory limits, no liveness or readiness probes, images using the `latest` tag, privilege escalation allowed, and many more.

```
91  checkov-k8s:
92    name: Checkov Kubernetes Scan
93    runs-on: ubuntu-latest
94
95    steps:
96      - name: Checkout source code
97        uses: actions/checkout@v7
98
99      - name: Setup Python
100       uses: actions/setup-python@v7
101       with:
102         python-version: "3.13"
103
104     - name: Install Checkov
105       run: pip install checkov
```

Lines 91–105 Same shape as Semgrep: runner, checkout, Python 3.13, then `pip install checkov`.

```
107   - name: Run Checkov Kubernetes scan
108     run: |
109       checkov -d k8-manifests/qa \
110         --framework kubernetes \
111         --output json > checkov-k8s-report.json || true
```

Line 109 `checkov -d k8-manifests/qa` scans every file in that directory.

Line 110 `--framework kubernetes` restricts Checkov to its Kubernetes checks. Without it, Checkov would try every framework it knows.

Line 111 `--output json` prints the results as JSON to standard output, and `> checkov-k8s-report.json` redirects that output into a file. Checkov exits with code 1 when any check fails; `|| true` stops that from failing the step.

```
113   - name: Upload Checkov Kubernetes report
114     uses: actions/upload-artifact@v7
115     with:
116       name: checkov-k8s-report
117       path: checkov-k8s-report.json
118       if-no-files-found: warn
```

Lines 113–118 Upload `checkov-k8s-report.json` as an artifact.

WHAT THE SCAN ACTUALLY COVERS

It scans the manifests as they are in the commit — that is, with the image tag from the *previous* deployment. The image line is rewritten later, after the scan. That is fine: Checkov cares about the structure of the manifest (security context, limits, probes), not which tag it points at.

Job 4 — checkov-dockerfile `lines 120–147`

The same tool, pointed at our Dockerfiles. Checkov's Dockerfile checks look for things like a missing `USER` instruction (so the container runs as root), no `HEALTHCHECK`, use of `ADD` instead of `COPY`, and `latest` base images.

```
120   checkov-dockerfile:
121     name: Checkov Dockerfile Scan
122     runs-on: ubuntu-latest
123
124     steps:
125       - name: Checkout source code
126         uses: actions/checkout@v7
127
128       - name: Setup Python
129         uses: actions/setup-python@v7
130         with:
131           python-version: "3.13"
132
133       - name: Install Checkov
134         run: pip install checkov
135
136       - name: Run Checkov Dockerfile scan
137         run: |
138           checkov -d . \
139             --framework dockerfile \
140             --output json > checkov-dockerfile-report.json || true
141
142       - name: Upload Checkov Dockerfile report
143         uses: actions/upload-artifact@v7
144         with:
145           name: checkov-dockerfile-report
146           path: checkov-dockerfile-report.json
147           if-no-files-found: warn
```

Lines 120–134 Identical setup to the Kubernetes job: runner, checkout, Python, install Checkov.

Line 138 `checkov -d .` scans from the repository root, so it finds both `backend/Dockerfile` and `frontend/Dockerfile` in one pass.

Line 139 `--framework dockerfile` — only Dockerfile checks. Even though we scan the whole repository, the Kubernetes YAML is ignored here because that framework is not selected.

Line 140 JSON to a file, `|| true` to keep the job green.

Lines 142–147 Upload it as `checkov-dockerfile-report`.

WHY TWO CHECKOV JOBS INSTEAD OF ONE

One Checkov run with both frameworks would work. Splitting them gives two separate reports with clear names, two separate entries in the job graph, and lets both run in parallel. When a reviewer asks "is the Dockerfile clean?", there is exactly one artifact to open.

Job 5 — backend-test `lines 149–190`

This job proves the backend compiles and its tests pass, then scans its dependencies for known vulnerabilities.

```
149  backend-test:
150    name: Backend - Maven Test + Trivy FS
151    runs-on: ubuntu-latest
152
153    steps:
154      - name: Checkout source code
155        uses: actions/checkout@v7
156
157      - name: Setup Java 21
158        uses: actions/setup-java@v6
159        with:
160          distribution: temurin
161          java-version: "21"
162          cache: maven
163          cache-dependency-path: backend/pom.xml
```

Lines 157–158 `actions/setup-java` installs a JDK and sets `JAVA_HOME`.

Line 160 `distribution: temurin` — the Eclipse Temurin build of OpenJDK, a widely used free distribution.

Line 161 `java-version: "21"` — Java 21, the long-term-support release our backend targets.

Line 162 `cache: maven` turns on caching of the local Maven repository (`~/.m2/repository`). The first run downloads every dependency; later runs restore them from the cache in seconds.

Line 163 `cache-dependency-path: backend/pom.xml` — the cache key is built from a hash of this file. Change a dependency in the POM and the key changes, so a fresh cache is built. The `pom.xml` is not at the repository root, so we have to point at it.

```
165    - name: Run backend tests
166      working-directory: backend
167      run: mvn -B clean verify
```

Line 166 `working-directory: backend` — run this step from inside `backend/`, where the `pom.xml` is.

Line 167 `mvn -B clean verify`. `-B` is batch mode: no interactive prompts and no animated download progress, which keeps CI logs readable. `clean` deletes any old `target/` folder. `verify` runs the Maven lifecycle up to and including verify — compile, run unit tests, package the JAR, run integration tests and any verification plugins. If any test fails, Maven exits non-zero and **this step fails the job** — this is the one hard gate in the stage-1 jobs.

```
169   - name: Install Trivy
170     uses: aquasecurity/setup-trivy@v0.2.6
171     with:
172       version: latest
173       cache: true
```

Line 170 `aquasecurity/setup-trivy` installs the Trivy CLI.

Line 172 `version: latest` — install the newest Trivy release.

Line 173 `cache: true` — cache the Trivy installation between runs so later runs set it up faster.

```
175   - name: Trivy FS scan - backend
176     run: |
177       trivy fs ./backend \
178         --offline-scan \
179         --format json \
180         --output backend-trivy-fs-report.json \
181         --severity HIGH,CRITICAL \
182         --ignore-unfixed \
183         --exit-code 0
```

Line 177 `trivy fs ./backend` scans a folder on disk (as opposed to `trivy image`, which scans a container image). In a Java project it reads `pom.xml` and the built JAR from `target/` to find dependencies, then compares them against its vulnerability database.

Line 178 `--offline-scan` stops Trivy from calling Maven Central to look up extra metadata about JAR files. The vulnerability database is still downloaded; only those per-artifact lookups are skipped, which makes the scan faster and avoids rate limits.

Line 179 `--format json` — JSON output.

Line 180 `--output backend-trivy-fs-report.json` — the file to write.

Line 181 `--severity HIGH,CRITICAL` — only report the two most serious severity levels, so the report is not drowned in LOW and MEDIUM findings.

Line 182 `--ignore-unfixed` — skip vulnerabilities that have no fixed version available yet. If there is nothing we can upgrade to, the finding is not actionable today.

Line 183 `--exit-code 0` — report only; never fail the step.

```
185   - name: Upload backend Trivy report
186     uses: actions/upload-artifact@v7
187     with:
188       name: backend-trivy-fs-report
189       path: backend-trivy-fs-report.json
190       if-no-files-found: warn
```

Lines 185–190 Upload the report as `backend-trivy-fs-report`.

Job 6 — frontend-build `lines 192–235`

The frontend's twin of the previous job: scan the dependencies, then prove the frontend builds.

```yaml
192  frontend-build:
193    name: Frontend - Trivy FS + Build
194    runs-on: ubuntu-latest
195
196    steps:
197      - name: Checkout source code
198        uses: actions/checkout@v7
199
200      - name: Install Trivy
201        uses: aquasecurity/setup-trivy@v0.2.6
202        with:
203          version: latest
204          cache: true
205
206      - name: Trivy FS scan - frontend
207        run: |
208          trivy fs ./frontend \
209            --format json \
210            --output frontend-trivy-fs-report.json \
211            --severity HIGH,CRITICAL \
212            --ignore-unfixed \
213            --exit-code 0
```

Lines 200–204 Install Trivy, exactly as in the backend job.

Line 208 `trivy fs ./frontend` — scan the frontend folder. Notice the order: this runs **before** `npm ci`, so there is no `node_modules` yet. Trivy reads `package-lock.json`, which lists every direct and transitive dependency with its exact version. That is enough for an accurate scan and much faster than walking thousands of files in `node_modules`.

Lines 209–213 Same flags as the backend scan: JSON, to a named file, HIGH and CRITICAL only, ignore unfixed, never fail. (There is no `--offline-scan` here; that flag only matters for Java artifacts.)

```yaml
215  - name: Upload frontend Trivy report
216    uses: actions/upload-artifact@v7
217    with:
218      name: frontend-trivy-fs-report
219      path: frontend-trivy-fs-report.json
220      if-no-files-found: warn
```

Lines 215–220 Upload as `frontend-trivy-fs-report`.

```
222   - name: Setup Node.js 22
223     uses: actions/setup-node@v6
224     with:
225       node-version: "22"
226       cache: npm
227       cache-dependency-path: frontend/package-lock.json
228
229   - name: Install frontend dependencies
230     working-directory: frontend
231     run: npm ci --no-audit --no-fund
232
233   - name: Build frontend
234     working-directory: frontend
235     run: npm run build
```

Lines 222–223 `actions/setup-node` installs Node.js.

Line 225 `node-version: "22"` — the Node 22 LTS line.

Line 226 `cache: npm` caches npm's download cache (`~/.npm`), not `node_modules`.

Line 227 `cache-dependency-path: frontend/package-lock.json` — the cache key comes from the lock file; it lives in `frontend/`, not the root.

Line 231 `npm ci` is the CI-friendly install: it deletes any existing `node_modules`, installs **exactly** the versions in `package-lock.json`, and fails if `package.json` and the lock file disagree. `--no-audit` skips npm's own vulnerability audit (Trivy already covered that) and `--no-fund` hides the funding messages. Both just make the step faster and quieter.

Line 235 `npm run build` runs the `build` script from `package.json` and produces the production bundle (the `dist/` folder that SonarQube later excludes). If the code does not compile, this step fails the job.

> **WHY BUILD HERE IF THE DOCKERFILE BUILDS IT AGAIN?**
> This build is a fast correctness check that runs in parallel with everything else. If the frontend is broken, we find out in stage 1 — and because `docker` needs `frontend-build`, we never waste time building an image that would fail. The bundle produced here is thrown away; the image is built from scratch by its own Dockerfile.

Job 7 — sonarqube `lines 237–283`

SonarQube analyses code quality and security together: bugs, vulnerabilities, security hotspots, code smells, duplication, and test coverage when reports are available. Its **quality gate** is a pass/fail verdict on the analysis, configured on the SonarQube server.

```yaml
  sonarqube:
    name: SonarQube Analysis
    runs-on: ubuntu-latest
    needs:
      - backend-test
      - frontend-build

    steps:
      - name: Checkout source code
        uses: actions/checkout@v7
        with:
          fetch-depth: 0
```

Lines 240–242 `needs: [backend-test, frontend-build]` — wait until both build jobs succeed. There is no point analysing code that does not compile.

Lines 247–248 `fetch-depth: 0` fetches the full Git history instead of one commit. SonarQube uses history for blame information (who last changed each line) and to work out what counts as “new code” for the quality gate. A shallow clone produces a warning and weaker new-code detection.

```yaml
  - name: Setup Java 21
    uses: actions/setup-java@v6
    with:
      distribution: temurin
      java-version: "21"
      cache: maven
      cache-dependency-path: backend/pom.xml

  - name: Compile backend for SonarQube
    working-directory: backend
    run: mvn -B clean verify
```

Lines 250–256 Install Java 21 with the Maven cache — the same as in `backend-test`.

Lines 258–260 Build the backend again with `mvn -B clean verify`. This job runs on a brand-new machine, so the compiled classes from `backend-test` do not exist here. SonarQube’s Java analyser needs compiled `.class` files to do its deeper analysis, so we must compile.

```yaml
  - name: SonarQube scan
    uses: SonarSource/sonarqube-scan-action@v8.2.1
    with:
      args: >
        -Dsonar.projectKey=${{ vars.SONAR_PROJECT_KEY || github.event.repository.name }}
        -Dsonar.projectName=${{ github.event.repository.name }}
        -Dsonar.sources=backend/src/main/java,frontend/src
        -Dsonar.tests=backend/src/test/java
        -Dsonar.java.binaries=backend/target/classes
        -Dsonar.exclusions=frontend/node_modules/**,frontend/dist/**
    env:
      SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
      SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
```

Line 263 `SonarSource/sonarqube-scan-action` runs the SonarScanner CLI against the SonarQube server.

Line 265 `args: >` — the `>` is YAML's "folded" style: the following lines are joined with spaces into a single line, so each `-D` flag becomes one long argument string.

Line 266 `-Dsonar.projectKey=` — the project's unique key on the server. The expression `${{ vars.SONAR_PROJECT_KEY || github.event.repository.name }}` uses the repository variable if it is set, otherwise falls back to the repository's name. `||` here is GitHub's expression "or", not a shell operator.

Line 267 `-Dsonar.projectName=` — the display name in the SonarQube UI, set to the repository name.

Line 268 `-Dsonar.sources=` — where production code lives: the backend Java sources and the frontend `src` folder, comma-separated. One SonarQube project covers both.

Line 269 `-Dsonar.tests=` — where test code lives, so SonarQube treats it as tests rather than production code.

Line 270 `-Dsonar.java.binaries=` — where the compiled classes are, produced by the Maven step above.

Line 271 `-Dsonar.exclusions=` — skip `node_modules` and the built `dist/` bundle. Analysing third-party or generated code would only add noise and time.

Lines 272–274 Step-level `env:` passes the two secrets to the scanner. `SONAR_TOKEN` authenticates, `SONAR_HOST_URL` says which server. Secrets are masked as `***` in logs.

```
276   - name: SonarQube Quality Gate
277     uses: SonarSource/sonarqube-quality-gate-action@v1.2.0
278     continue-on-error: true
279     with:
280       pollingTimeoutSec: 600
281     env:
282       SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
283       SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
```

Line 277 `sonarqube-quality-gate-action` waits for the server to finish processing the analysis and reads the quality gate result. It finds the analysis through the `report-task.txt` file that the scan step left in `.scannerwork/`.

Line 278 `continue-on-error: true` — if the gate fails, the step is marked as failed with a warning, but the job still counts as successful. Combined with the fact that nothing `needs` this job, the gate is informational.

Line 280 `pollingTimeoutSec: 600` — wait up to ten minutes for the server to compute the result before giving up.

Lines 281–283 Same two secrets, so the action can query the server.

Job 8 — docker lines 285–424

This is the heart of the pipeline. It logs in to AWS without any stored keys, builds both images, scans them, generates SBOMs, and only then pushes them to ECR. It also publishes the final image names as outputs so later jobs know exactly what was built.

```
285   docker:
286     name: Build, Scan and Push Backend + Frontend to ECR
287     runs-on: ubuntu-latest
288
289     permissions:
290       contents: read
291       id-token: write
292
293     needs:
294       - gitleaks-scan
295       - semgrep-scan
296       - checkov-k8s
297       - checkov-dockerfile
298       - backend-test
299       - frontend-build
300
301     outputs:
302       image_tag: ${{ steps.image-meta.outputs.image_tag }}
303       backend_image: ${{ steps.image-meta.outputs.backend_image }}
304       frontend_image: ${{ steps.image-meta.outputs.frontend_image }}
```

Lines 289–291 Job-level `permissions` **replace** the workflow-level ones for this job — they are not merged. So `contents: read` has to be repeated, otherwise checkout would lose access. `id-token: write` lets the job request an OIDC token from GitHub; without it, the AWS login step fails with an error about the token.

Lines 293–299 `needs:` — all six stage-1 jobs must succeed. If tests fail or the frontend does not build, no image is built. (The scanners never fail, so in practice the real gates are the Maven tests and the npm build.)

Lines 301–304 `outputs:` publishes three values from this job for other jobs to read. Each one points at an output of the step with `id: image-meta` (defined below). Later jobs read them as `needs.docker.outputs.backend_image` and so on.

```
306     steps:
307       - name: Checkout source code
308         uses: actions/checkout@v7
309
310       - name: Show GitHub Repository IDs
311         run: |
312           echo "Repository: ${{ github.repository }}"
313           echo "Repository ID: ${{ github.repository_id }}"
314           echo "Owner: ${{ github.repository_owner }}"
315           echo "Owner ID: ${{ github.repository_owner_id }}"
316           echo "Ref: ${{ github.ref }}"
```

Lines 307–308 Check out the code; the Dockerfiles and build contexts are in it.

Lines 310–316 Print the repository name and numeric ID, the owner name and numeric ID, and the Git ref. This is purely diagnostic. It is very handy when you are writing or debugging the IAM trust policy: the OIDC token can carry these numeric IDs, and unlike names they do not change if the repository or organisation is renamed.

```
318   - name: Configure AWS credentials from GitHub OIDC
319     uses: aws-actions/configure-aws-credentials@v6.3.0
320     with:
321       role-to-assume: ${{ secrets.AWS_QA_ROLE_TO_ASSUME }}
322       aws-region: ${{ env.AWS_REGION }}
323
324   - name: Login to Amazon ECR
325     id: login-ecr
326     uses: aws-actions/amazon-ecr-login@v2
```

Line 319 `aws-actions/configure-aws-credentials` performs the OIDC login. Behind the scenes it asks GitHub for a signed token for this job, then calls AWS STS `AssumeRoleWithWebIdentity` with it.

Line 321 `role-to-assume:` — the ARN of the QA role, read from the secret. AWS checks the token against that role's trust policy (the one we saw in Chapter 2).

Line 322 `aws-region:` — taken from the workflow-level `env`. The step exports `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` and `AWS_SESSION_TOKEN` for the rest of the job — temporary credentials that expire on their own.

Lines 324–326 `aws-actions/amazon-ecr-login` runs `docker login` against our ECR registry using those credentials. `id: login-ecr` lets later steps read its output `registry` — the registry hostname, which looks like `<account-id>.dkr.ecr.ap-south-1.amazonaws.com`.

Naming the images

```
328   - name: Set image names
329     id: image-meta
330     shell: bash
331     run: |
332       REGISTRY="${{ steps.login-ecr.outputs.registry }}"
333       SHORT_SHA="${GITHUB_SHA::7}"
334       IMAGE_TAG="qa-${GITHUB_RUN_NUMBER}-${GITHUB_RUN_ATTEMPT}-${SHORT_SHA}"
335
336       BACKEND_IMAGE="$REGISTRY/$BACKEND_REPO:$IMAGE_TAG"
337       FRONTEND_IMAGE="$REGISTRY/$FRONTEND_REPO:$IMAGE_TAG"
338
339       echo "REGISTRY=$REGISTRY" >> "$GITHUB_ENV"
340       echo "IMAGE_TAG=$IMAGE_TAG" >> "$GITHUB_ENV"
341       echo "BACKEND_IMAGE=$BACKEND_IMAGE" >> "$GITHUB_ENV"
342       echo "FRONTEND_IMAGE=$FRONTEND_IMAGE" >> "$GITHUB_ENV"
343
344       echo "image_tag=$IMAGE_TAG" >> "$GITHUB_OUTPUT"
345       echo "backend_image=$BACKEND_IMAGE" >> "$GITHUB_OUTPUT"
346       echo "frontend_image=$FRONTEND_IMAGE" >> "$GITHUB_OUTPUT"
```

Line 329 `id: image-meta` — the ID the job `outputs:` block refers to.

Line 330 `shell: bash` — run with Bash explicitly. GitHub then uses `bash --noprofile --norc -eo pipefail`, so any failing command stops the step.

Line 332 `REGISTRY=` — the registry hostname from the ECR login step.

Line 333 `SHORT_SHA="${GITHUB_SHA::7}"` — Bash substring syntax: the first 7 characters of the full 40-character commit SHA, the same short form Git shows.

Line 334 `IMAGE_TAG="qa-${GITHUB_RUN_NUMBER}-${GITHUB_RUN_ATTEMPT}-${SHORT_SHA}"` builds a tag like `qa-42-1-a1b2c3d`. The `qa-` prefix says which environment built it. `GITHUB_RUN_NUMBER` increases by one with every run of this workflow. `GITHUB_RUN_ATTEMPT` is 1 for the first attempt and goes up when you press **Re-run** — so a re-run gets a new tag and never overwrites an image that might already be deployed. The short SHA ties the image to the exact commit it was built from.

Lines 336–337 The full image references: `registry/repository:tag` for each of the two images.

Lines 339–342 Appending `NAME=value` lines to the file at `$GITHUB_ENV` makes them **environment variables for all later steps in this job**. That is why later steps can simply use `$BACKEND_IMAGE`.

Lines 344–346 Appending to `$GITHUB_OUTPUT` makes them **outputs of this step**, readable as `steps.image-meta.outputs.backend_image`. The job's `outputs:` block (lines 301–304) then lifts them to the job level so other jobs can use them. `GITHUB_ENV` is for the same job, `GITHUB_OUTPUT` is for passing data on.

Building the images

```
348   - name: Set up Docker Buildx
349     uses: docker/setup-buildx-action@v4
350
351   - name: Build backend image
352     uses: docker/build-push-action@v7
353     with:
354       context: ./backend
355       load: true
356       push: false
357       tags: ${{ steps.image-meta.outputs.backend_image }}
358       cache-from: type=gha,scope=projectops-backend
359       cache-to: type=gha,mode=max,scope=projectops-backend
360
361   - name: Build frontend image
362     uses: docker/build-push-action@v7
363     with:
364       context: ./frontend
365       load: true
366       push: false
367       tags: ${{ steps.image-meta.outputs.frontend_image }}
368       cache-from: type=gha,scope=projectops-frontend
369       cache-to: type=gha,mode=max,scope=projectops-frontend
```

Lines 348–349 `docker/setup-buildx-action` creates a Buildx builder. Buildx is Docker's BuildKit-based builder; we need it for the GitHub Actions cache backend used below.

Line 352 `docker/build-push-action` builds an image with Buildx.

Line 354 `context: ./backend` — the build context is the `backend/` folder, and by default the action looks for `backend/Dockerfile`.

Line 355 `load: true` — after building, load the image into the runner's local Docker engine. Buildx otherwise keeps results inside its builder, where `docker push` and the scanners cannot see them.

Line 356 `push: false` — do **not** push yet. We want to scan first and push only afterwards.

Line 357 `tags:` — tag the image with the full ECR name computed above.

Line 358 `cache-from: type=gha,scope=projectops-backend` — reuse layers saved in the GitHub Actions cache from previous runs. If only application code changed, the base image and dependency layers come from the cache and the build takes a fraction of the time.

Line 359 `cache-to: type=gha,mode=max,scope=projectops-backend` — save layers back into the cache. `mode=max` caches every intermediate layer, including those from earlier stages of a multi-stage Dockerfile, not just the final image's layers. `scope` keeps the backend and frontend caches separate so they do not overwrite each other.

Lines 361–369 Exactly the same for the frontend, with its own context, tag and cache scope.

Scanning the images and generating SBOMs

```
371   - name: Install Trivy
372     uses: aquasecurity/setup-trivy@v0.2.6
373     with:
374       version: latest
375       cache: true
376
377   - name: Scan backend image
378     run: |
379       trivy image "$BACKEND_IMAGE" \
380         --format json \
381         --output backend-image-trivy-report.json \
382         --severity HIGH,CRITICAL \
383         --ignore-unfixed \
384         --exit-code 0
385
386   - name: Scan frontend image
387     run: |
388       trivy image "$FRONTEND_IMAGE" \
389         --format json \
390         --output frontend-image-trivy-report.json \
391         --severity HIGH,CRITICAL \
392         --ignore-unfixed \
393         --exit-code 0
394
395   - name: Upload image scan reports
396     uses: actions/upload-artifact@v7
397     with:
398       name: trivy-image-reports
399       path: |
400         backend-image-trivy-report.json
401         frontend-image-trivy-report.json
402       if-no-files-found: warn
```

Lines 371–375 Install Trivy (each job is a new machine, so it is not there from the earlier jobs).

Lines 377–384 `trivy image "$BACKEND_IMAGE"` scans the image we just built and loaded locally. This is different from the filesystem scan: it also sees the **operating-system packages** in the base image (for example Debian or Alpine packages) and everything copied in during the build. The flags mean the same as before: JSON, a named file, HIGH and CRITICAL only, ignore unfixed, never fail.

Lines 386–393 The same scan for the frontend image.

Lines 395–402 Upload both image reports as one artifact called `trivy-image-reports`. The `|` under `path:` allows several paths, one per line.

```
404   - name: Generate backend image SBOM
405     uses: anchore/sbom-action@v0
406     with:
407       image: ${{ steps.image-meta.outputs.backend_image }}
408       format: spdx-json
409       output-file: backend-image-sbom.spdx.json
410       upload-artifact: true
411
412   - name: Generate frontend image SBOM
413     uses: anchore/sbom-action@v0
414     with:
415       image: ${{ steps.image-meta.outputs.frontend_image }}
416       format: spdx-json
417       output-file: frontend-image-sbom.spdx.json
418       upload-artifact: true
```

Line 405 `anchore/sbom-action` runs Syft to produce a **Software Bill of Materials** — a complete list of every package and library in the image, with versions and licences.

Line 407 `image:` — which image to inventory; here the step output from `image-meta`. Syft reads it from the local Docker engine, since we loaded it there.

Line 408 `format: spdx-json` — SPDX is an open, standard SBOM format that many security and compliance tools understand.

Line 409 `output-file:` — where to write it.

Line 410 `upload-artifact: true` — the action uploads the SBOM as a workflow artifact by itself; no separate upload step needed.

Lines 412–418 The same for the frontend image.

> **WHY AN SBOM WHEN WE ALREADY SCANNED?**
> A Trivy report is a snapshot of what was known to be vulnerable on the day we built. An SBOM is a record of what is *in* the image. When a new CVE is announced next month, we can check the stored SBOMs to see which already-deployed images contain the affected library — without rebuilding or even pulling anything. Many customers and regulators now ask for SBOMs as part of software supply-chain security.

Pushing to ECR

```
420   - name: Push backend image to ECR
421     run: docker push "$BACKEND_IMAGE"
422
423   - name: Push frontend image to ECR
424     run: docker push "$FRONTEND_IMAGE"
```

Lines 420–421 `docker push "$BACKEND_IMAGE"` uploads the backend image to ECR. The ECR login from earlier in this job is still valid.

Lines 423–424 The same for the frontend. These are the last two steps, so an image only reaches ECR after it has been built, scanned and inventoried.

Job 9 — update-qa-manifests `lines 426–472`

Now that the new images are in ECR, this job writes their names into the QA manifests and commits the change back to the `qa` branch. After this job, Git says exactly what QA is about to run.

```
426   update-qa-manifests:
427     name: Update QA Manifests with ECR Images
428     runs-on: ubuntu-latest
429
430     permissions:
431       contents: write
432
433     needs: docker
434
435     steps:
436       - name: Checkout QA branch
437         uses: actions/checkout@v7
438         with:
439           ref: qa
440           token: ${{ secrets.GITHUB_TOKEN }}
```

Lines 430–431 `contents: write` lets the `GITHUB_TOKEN` push a commit. No `id-token` here — this job never talks to AWS.

Line 433 `needs: docker` — only after the images are pushed. A single job can be written without the list brackets.

Line 439 `ref: qa` — check out the **current tip** of the `qa` branch rather than the commit that triggered the run. That way we commit on top of the latest state of the branch.

Line 440 `token: ${{ secrets.GITHUB_TOKEN }}` — the token checkout stores in the local Git config, so that the `git push` later is authenticated. This is the default anyway; writing it out makes the intent clear.

```
442   - name: Update backend and frontend image references
443     env:
444       BACKEND_IMAGE: ${{ needs.docker.outputs.backend_image }}
445       FRONTEND_IMAGE: ${{ needs.docker.outputs.frontend_image }}
446     run: |
447       sed -Ei "s|^([[:space:]]*)image:.*|\1image: ${BACKEND_IMAGE}|" \
448         k8-manifests/qa/backend.yaml
449
450       sed -Ei "s|^([[:space:]]*)image:.*|\1image: ${FRONTEND_IMAGE}|" \
451         k8-manifests/qa/frontend.yaml
452
453       echo "Backend image:"
454       grep -m1 "image:" k8-manifests/qa/backend.yaml
455
456       echo "Frontend image:"
457       grep -m1 "image:" k8-manifests/qa/frontend.yaml
```

Lines 443–445 Step-level `env:` reads the image names from the `docker` job's outputs and exposes them as shell variables.

Line 447 The `sed` command that rewrites the image line. Piece by piece:

- `-E` turns on extended regular expressions, so `( )` creates a capture group without backslashes.
- `-i` edits the file in place instead of printing the result.
- `s|pattern|replacement|` is a substitution. We use `|` as the separator instead of the usual `/`, because the image name itself contains slashes.
- `^([[:space:]]*)image:.*` matches a line that starts with any amount of whitespace, then `image:`, then anything. The whitespace is captured as group 1.
- `\1image: ${BACKEND_IMAGE}` writes the same indentation back (`\1`), then `image:` and the new image name. Keeping the indentation keeps the YAML valid.

Line 448 The file to edit: the QA backend manifest.

Lines 450–451 The same for the frontend manifest with the frontend image.

Lines 453–457 Print the first `image:` line of each file. `grep -m1` stops after the first match. It is a quick visual check in the log that the substitution worked.

> **SED REPLACES EVERY IMAGE: LINE IN THE FILE**
> The pattern is not limited to one container. If a manifest ever gains a second container — a sidecar, or an init container — its `image:` line will be overwritten with the backend image too. While each file has exactly one container this is fine. If that changes, switch to a more targeted tool such as `yq`, or `kubectl set image` against a specific container name.

```
459   - name: Commit updated QA manifests
460     run: |
461       git config user.name "github-actions[bot]"
462       git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
463
464       git add \
465         k8-manifests/qa/backend.yaml \
466         k8-manifests/qa/frontend.yaml
467
468       git commit \
469         -m "Update QA ECR images to ${{ needs.docker.outputs.image_tag }} [skip ci]" \
470         || echo "No changes to commit"
471
472       git push origin qa
```

Lines 461–462 Set the commit author to the official GitHub Actions bot. `41898282` is that bot account's user ID; using its no-reply email means GitHub shows the commit with the bot's avatar.

Lines 464–466 Stage only the two files we changed.

Lines 468–470 Commit with a message that includes the new tag, so the Git log reads like a deployment history. `[skip ci]` in the message tells GitHub not to start workflows for this commit. If nothing changed (for example a re-run produced the same file content), `git commit` exits with an error; `|| echo "No changes to commit"` turns that into a harmless log line.

Line 472 `git push origin qa` pushes the commit. When there was nothing to commit, this is a no-op that reports "Everything up-to-date".

> **TWO WAYS THIS PUSH CAN FAIL**
> If someone pushed to `qa` between this job's checkout and its push, the push is rejected as non-fast-forward and the job fails — re-running fixes it. And if `qa` has branch protection that requires pull requests, the bot cannot push directly; you would need to allow GitHub Actions to bypass the rule, or push with a GitHub App token that is on the bypass list.

Job 10 — deploy-to-k8s `lines 474–538`

The last job applies the updated manifests to the `qa` namespace on EKS and waits until the new pods are actually up.

```
474   deploy-to-k8s:
475     name: Deploy Backend + Frontend + Ingress to QA EKS
476     runs-on: ubuntu-latest
477
478     permissions:
479       contents: read
480       id-token: write
481
482     needs: update-qa-manifests
483
484     steps:
485       - name: Checkout source code
486         uses: actions/checkout@v7
487         with:
488           ref: qa
489
490       - name: Configure AWS credentials from GitHub OIDC
491         uses: aws-actions/configure-aws-credentials@v6.3.0
492         with:
493           role-to-assume: ${{ secrets.AWS_QA_ROLE_TO_ASSUME }}
494           aws-region: ${{ env.AWS_REGION }}
```

Lines 478–480 `id-token: write` again — this job logs in to AWS on its own. Credentials from the `docker` job lived on a different machine that no longer exists.

Line 482 `needs: update-qa-manifests` — deploy only after the new tags are in Git.

Lines 485–488 Check out `ref: qa`. This is important: the commit that triggered the run has the **old** image tags. The bot's new commit is at the tip of `qa`, so we must check out the branch to deploy what we just built.

Lines 490–494 OIDC login with the QA role, exactly as in the `docker` job.

```
496   - name: Setup kubectl
497     uses: azure/setup-kubectl@v4
498
499   - name: Generate kubeconfig for EKS
500     run: |
501       aws eks update-kubeconfig \
502         --region "$AWS_REGION" \
503         --name "$EKS_CLUSTER_NAME"
```

Lines 496–497 `azure/setup-kubectl` installs `kubectl`. (It is published by Microsoft but works with any Kubernetes cluster.)

Lines 499–503 `aws eks update-kubeconfig` writes `~/.kube/config` for `my-cluster` in `ap-south-1`. It calls `eks:DescribeCluster` to fetch the API server address and certificate. The kubeconfig it writes does not store a token; instead, each `kubectl` call runs `aws eks get-token` to get a fresh one from the current AWS credentials.

```
505   - name: Verify AWS and namespace access
506     run: |
507       aws sts get-caller-identity
508
509       kubectl auth can-i create deployments \
510         -n "$K8S_NAMESPACE"
511
512       kubectl auth can-i create services \
513         -n "$K8S_NAMESPACE"
514
515       kubectl auth can-i create ingresses.networking.k8s.io \
516         -n "$K8S_NAMESPACE"
```

Line 507 `aws sts get-caller-identity` prints the account and the ARN of the assumed role. If the wrong role was assumed, this is where you see it.

Lines 509–516 `kubectl auth can-i create <resource> -n qa` asks the cluster whether we may create Deployments, Services and Ingresses in `qa`, and prints `yes` or `no`. `ingresses.networking.k8s.io` is the fully qualified resource name. When the answer is `no`, the command also exits with code 1, so the step fails right here with a clear message instead of halfway through a deploy.

```
518   - name: Deploy QA application
519     run: |
520       kubectl apply -f k8-manifests/qa/backend.yaml
521       kubectl apply -f k8-manifests/qa/frontend.yaml
522       kubectl apply -f k8-manifests/qa/ingress.yaml
```

Lines 520–522 `kubectl apply -f` sends each manifest to the cluster. `apply` is declarative: it creates the object if it does not exist and updates it if it does. Because the image changed, Kubernetes starts a rolling update of the Deployment — new pods with the new image come up, old ones are removed.

```
524   - name: Verify rollout
525     run: |
526       kubectl rollout status deployment/backend \
527         -n "$K8S_NAMESPACE" \
528         --timeout=300s
529
530       kubectl rollout status deployment/frontend \
531         -n "$K8S_NAMESPACE" \
532         --timeout=300s
```

Lines 526–528 `kubectl rollout status deployment/backend` waits until the backend rollout completes: all replicas updated and available. `--timeout=300s` gives up after five minutes and fails the step. Without this step the pipeline would go green the moment the manifest was accepted, even if the new pods were crash-looping.

Lines 530–532 The same wait for the frontend.

```
534   - name: Show QA resources
535     run: |
536       kubectl get pods -n "$K8S_NAMESPACE" -o wide
537       kubectl get services -n "$K8S_NAMESPACE"
538       kubectl get ingress -n "$K8S_NAMESPACE"
```

Lines 536–538 Print the pods (with `-o wide`, showing node and pod IP), the services and the ingress in `qa`. The ingress output includes the load balancer address, which is the URL testers use.

AFTER A GREEN RUN

The QA environment is now running images tagged `qa-<run>-<attempt>-<sha>`, the `qa` branch has a bot commit recording that tag, and the run's summary page holds every scan report and both SBOMs. This is the point where the testing team takes over. Production is not touched until someone decides to promote.

CHAPTER 4 · PROD-CD.YAML

The Production Pipeline, Line by Line

The production pipeline is deliberately boring. It builds nothing and scans nothing — all of that already happened in QA. It promotes, records and deploys, and it checks its own access at every step before doing anything irreversible.

The workflow header `lines 1–19`

```yaml
name: Enterprise Production CD Pipeline

on:
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: production
  cancel-in-progress: false

env:
  AWS_REGION: ap-south-1
  EKS_CLUSTER_NAME: my-cluster
  K8S_NAMESPACE: prod

  BACKEND_REPO: projectops-backend
  FRONTEND_REPO: projectops-frontend
```

Line 1 The name shown in the Actions tab.

Lines 3–4 `workflow_dispatch` is the **only** trigger. No push, no schedule, no pull request can start a production deployment — a person has to open the Actions tab, choose this workflow, pick the branch from the dropdown (use `main`) and press **Run workflow**. That human click is our release decision.

Lines 6–7 Read-only `GITHUB_TOKEN` by default, raised per job where needed — the same least-privilege pattern as QA.

Lines 9–11 `concurrency` with a fixed group name, `production`, so every production run shares one group no matter which branch it was started from. `cancel-in-progress: false` is the important difference from QA: a new run **waits** for the running one to finish instead of cancelling it. Killing a production deployment halfway through a rollout is far worse than waiting a few minutes. (GitHub keeps at most one run waiting in the queue; if you start a third while one is running and one is waiting, the waiting one is replaced by the newest.)

Lines 13–19 The same shared variables as QA, except `K8S_NAMESPACE` is `prod`. The repository names are the same: QA and production tags live in the same two ECR repositories.

How the three jobs fit together

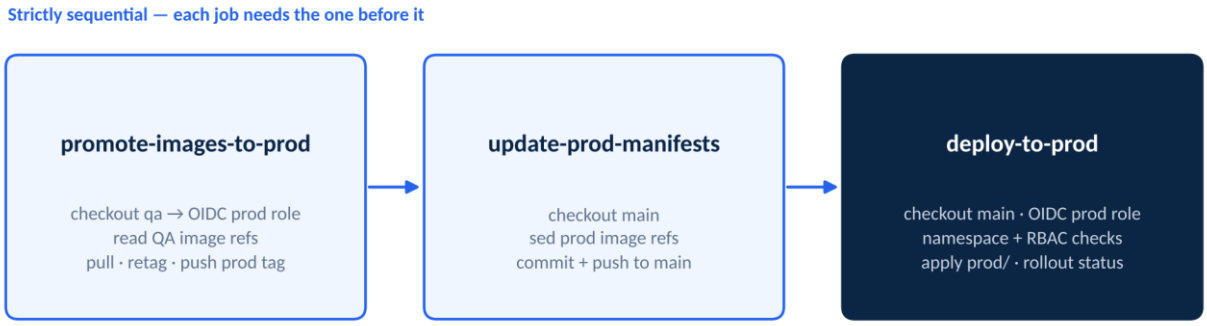


Figure 3 — prod-cd.yaml runs three jobs strictly in order.

Each job `needs` the one before it, so they run one after another. If the promotion fails, the manifests are not touched; if the manifest update fails, nothing is deployed. Each job leaves the system in a consistent state.

Job 1 — promote-images-to-prod `lines 27–197`

This job finds out which images QA is running, gives those exact images production tags, and pushes the tags to ECR. It never runs `docker build`.

```
24  # ============================================================
25  # 1. PROMOTE TESTED QA IMAGES TO PROD
26  # ============================================================
27  promote-images-to-prod:
28    name: Promote QA Images to Production
29    runs-on: ubuntu-latest
30
31    permissions:
32      contents: read
33      id-token: write
34
35    outputs:
36      prod_tag: ${{ steps.prod-meta.outputs.prod_tag }}
37      backend_prod_image: ${{ steps.prod-meta.outputs.backend_prod_image }}
38      frontend_prod_image: ${{ steps.prod-meta.outputs.frontend_prod_image }}
```

Lines 24–26 Comment banners. They have no effect; they make a long file easy to scan.

Lines 27–29 Job ID, display name, Ubuntu runner.

Lines 31–33 `contents: read` to check out code, `id-token: write` to log in to AWS with OIDC.

Lines 35–38 Three job outputs for the next jobs: the production tag and the two full production image names, all coming from the step with `id: prod-meta`.

Getting the QA branch and logging in

```
42  # ------------------------------------------------------------
43  # Checkout QA branch
44  # ------------------------------------------------------------
45  - name: Checkout QA branch
46    uses: actions/checkout@v7
47    with:
48      ref: qa
49      path: qa-source
50      fetch-depth: 0
```

Line 46 Check out the repository…

Line 48 …at `ref: qa`, because the QA manifests — and therefore the record of what QA is running — live on that branch.

Line 49 `path: qa-source` checks it out into a subfolder instead of the workspace root. That makes it obvious in every later command that we are reading from the QA copy (`qa-source/k8-manifests/qa/...`), and leaves the root free if another checkout is ever added.

Line 50 `fetch-depth: 0` fetches the full history. It is not strictly needed for reading one file, but it means `git log` would show the QA deployment history if we ever want to print it.

```
53  # ------------------------------------------------------------
54  # GitHub OIDC -> Production IAM Role
55  # ------------------------------------------------------------
56  - name: Configure AWS credentials from GitHub OIDC
57    uses: aws-actions/configure-aws-credentials@v6.3.0
58    with:
59      role-to-assume: ${{ secrets.AWS_PROD_ROLE_TO_ASSUME }}
60      aws-region: ${{ env.AWS_REGION }}
61
62
63  # ------------------------------------------------------------
64  # Verify assumed AWS identity
65  # ------------------------------------------------------------
66  - name: Verify AWS identity
67    run: aws sts get-caller-identity
68
69
70  # ------------------------------------------------------------
71  # Login to Amazon ECR
72  # ------------------------------------------------------------
73  - name: Login to Amazon ECR
74    id: login-ecr
75    uses: aws-actions/amazon-ecr-login@v2
```

Lines 56–60 OIDC login, this time with the **production** role from `AWS_PROD_ROLE_TO_ASSUME`. Its trust policy only accepts runs from `main`, so if someone dispatched this workflow from another branch, it fails right here.

Lines 66–67 `aws sts get-caller-identity` prints which role we actually got. In an audit, this log line shows that the production role — and not something else — performed the release.

Lines 73–75 Log in to ECR with the production role. `id: login-ecr` exposes the registry hostname for later steps.

Verifying and reading the QA manifests

```
78  # ------------------------------------------------------------
79  # Verify QA manifests
80  # ------------------------------------------------------------
81  - name: Verify QA manifests
82    shell: bash
83    run: |
84      set -euo pipefail
85
86      test -f qa-source/k8-manifests/qa/backend.yaml
87      test -f qa-source/k8-manifests/qa/frontend.yaml
88
89      echo "QA backend manifest found"
90      echo "QA frontend manifest found"
```

Line 82 `shell: bash` — run under Bash with `-eo pipefail`.

Line 84 `set -euo pipefail` — the standard "strict mode" for Bash, set explicitly inside the script. `-e` exits on the first failing command, `-u` treats any use of an undefined variable as an error (catching typos like `$BACKND_IMAGE`), and `-o pipefail` makes a pipeline fail if any command in it fails, not just the last one. Every step in this file starts with it.

Lines 86–87 `test -f <file>` succeeds if the file exists and is a regular file, and fails otherwise. With `set -e`, a missing manifest stops the job immediately with a clear point of failure.

Lines 89–90 Friendly confirmation in the log.

```
93  # ------------------------------------------------------------
94  # Read exact images that were tested in QA
95  # ------------------------------------------------------------
96  - name: Read QA image references
97    id: qa-images
98    shell: bash
99    run: |
100     set -euo pipefail
101
102     BACKEND_QA_IMAGE=$(awk \
103       '/^[[:space:]]*image:/ {print $2; exit}' \
104       qa-source/k8-manifests/qa/backend.yaml)
105
106     FRONTEND_QA_IMAGE=$(awk \
107       '/^[[:space:]]*image:/ {print $2; exit}' \
108       qa-source/k8-manifests/qa/frontend.yaml)
109
110     if [ -z "$BACKEND_QA_IMAGE" ]; then
111       echo "ERROR: Backend QA image not found"
112       exit 1
113     fi
114
115     if [ -z "$FRONTEND_QA_IMAGE" ]; then
116       echo "ERROR: Frontend QA image not found"
117       exit 1
118     fi
119
120     echo "Backend QA image: $BACKEND_QA_IMAGE"
121     echo "Frontend QA image: $FRONTEND_QA_IMAGE"
122
123     echo "backend_qa_image=$BACKEND_QA_IMAGE" >> "$GITHUB_OUTPUT"
124     echo "frontend_qa_image=$FRONTEND_QA_IMAGE" >> "$GITHUB_OUTPUT"
```

Line 97 `id: qa-images` so the next steps can read this step's outputs.

Lines 102–104 The `awk` command that extracts the image name. `/^[[:space:]]*image:/` selects the first line that starts with optional whitespace followed by `image:`. For that line, `{print $2; exit}` prints the second whitespace-separated field — the image reference itself, since field one is `image:` — and then stops reading. `$( ... )` captures the printed value into `BACKEND_QA_IMAGE`.

Lines 106–108 The same for the frontend manifest.

Lines 110–118 `[ -z "$VAR" ]` is true when the variable is empty. If `awk` found no `image:` line, we fail with an explicit error rather than trying to `docker pull` an empty name later and getting a cryptic message.

Lines 120–121 Log both QA images. This is the line to read when someone asks "what exactly did we promote?"

Lines 123–124 Publish them as step outputs.

READING THE TAG FROM GIT INSTEAD OF TAKING AN INPUT

The pipeline could have asked the person running it to type a tag. Reading it from the QA manifest removes a whole class of mistakes — typos, promoting a tag that never went to QA, promoting an older build by accident. Whatever is recorded as running in QA is, by definition, what gets promoted.

PROMOTE WHAT WAS SIGNED OFF — NOT WHAT CAME AFTER

The workflow reads the tip of `qa` at the moment it runs. If a new commit was deployed to QA after testers signed off, that newer, untested build is what gets promoted. In practice the team freezes `qa` between sign-off and promotion, or the release manager checks the logged QA image (lines 120–121) against the build that was approved.

Naming the production images

```
127   # ------------------------------------------------------------
128   # Generate immutable production image tag
129   # ------------------------------------------------------------
130   - name: Generate production image names
131     id: prod-meta
132     shell: bash
133     run: |
134       set -euo pipefail
135
136       REGISTRY="${{ steps.login-ecr.outputs.registry }}"
137
138       SHORT_SHA="${GITHUB_SHA::7}"
139
140       PROD_TAG="prod-${GITHUB_RUN_NUMBER}-${GITHUB_RUN_ATTEMPT}-${SHORT_SHA}"
141
142       BACKEND_PROD_IMAGE="$REGISTRY/$BACKEND_REPO:$PROD_TAG"
143       FRONTEND_PROD_IMAGE="$REGISTRY/$FRONTEND_REPO:$PROD_TAG"
144
145       echo "PROD_TAG=$PROD_TAG" >> "$GITHUB_ENV"
146       echo "BACKEND_PROD_IMAGE=$BACKEND_PROD_IMAGE" >> "$GITHUB_ENV"
147       echo "FRONTEND_PROD_IMAGE=$FRONTEND_PROD_IMAGE" >> "$GITHUB_ENV"
148
```

```
149       echo "prod_tag=$PROD_TAG" >> "$GITHUB_OUTPUT"
150       echo "backend_prod_image=$BACKEND_PROD_IMAGE" >> "$GITHUB_OUTPUT"
151       echo "frontend_prod_image=$FRONTEND_PROD_IMAGE" >> "$GITHUB_OUTPUT"
152
153       echo "Production tag: $PROD_TAG"
154       echo "Backend PROD image: $BACKEND_PROD_IMAGE"
155       echo "Frontend PROD image: $FRONTEND_PROD_IMAGE"
```

Line 136 The registry hostname from the ECR login.

Line 138 The first 7 characters of `GITHUB_SHA`. In a manually dispatched run, this is the commit at the tip of the branch the run was started from — normally `main` — not the commit the QA image was built from.

Line 140 `PROD_TAG` follows the same pattern as QA: `prod-<run number>-<attempt>-<short sha>`, for example `prod-17-1-9f8e7d6`. The run number here counts runs of *this* workflow, so production releases are numbered on their own.

Lines 142–143 The full production image names — same repositories, new tag.

Lines 145–147 Write them to `$GITHUB_ENV` for the rest of this job.

Lines 149–151 Write them to `$GITHUB_OUTPUT` so the job outputs (lines 35–38) can pass them to the next jobs.

Lines 153–155 Log all three values.

> **TRACING A PRODUCTION TAG BACK TO ITS BUILD**
> Because the SHA in the production tag is the `main` commit at release time, you trace a production image back to its build in two hops: the bot commit on `main` ("Promote images to prod-17-1-9f8e7d6") gives you the run, and that run's log (lines 120–121) shows the QA tag, whose SHA is the source commit. Even better, the image **digest** — the `sha256:` content hash shown in ECR — is identical for both tags, which is the strongest proof they are the same image.

Pull, retag, push — without rebuilding

```
158   # ------------------------------------------------------------
159   # Pull already tested QA images
160   # ------------------------------------------------------------
161   - name: Pull QA images from ECR
162     shell: bash
163     run: |
164       set -euo pipefail
165
166       docker pull "${{ steps.qa-images.outputs.backend_qa_image }}"
167       docker pull "${{ steps.qa-images.outputs.frontend_qa_image }}"
168
169
170   # ------------------------------------------------------------
171   # Retag SAME QA images for production
172   # No rebuilding
173   # ------------------------------------------------------------
174   - name: Retag QA images for production
175     shell: bash
176     run: |
177       set -euo pipefail
178
```

```
179       docker tag \
180         "${{ steps.qa-images.outputs.backend_qa_image }}" \
181         "$BACKEND_PROD_IMAGE"
182
183       docker tag \
184         "${{ steps.qa-images.outputs.frontend_qa_image }}" \
185         "$FRONTEND_PROD_IMAGE"
186
187
188   # ------------------------------------------------------------
189   # Push production tags to ECR
190   # ------------------------------------------------------------
191   - name: Push production images to ECR
192     shell: bash
193     run: |
194       set -euo pipefail
195
196       docker push "$BACKEND_PROD_IMAGE"
197       docker push "$FRONTEND_PROD_IMAGE"
```

Lines 166–167 `docker pull` downloads both QA-tested images from ECR to the runner. The production role needs pull permission on these repositories.

Lines 179–185 `docker tag <source> <target>` adds a second name to an existing local image. No layer is changed or copied — after this, the runner has one image with two tags, the QA one and the production one.

Lines 196–197 `docker push` uploads the production tags to ECR. Every layer already exists in the repository, so Docker skips them all and only writes the new tag pointing at the same manifest. The push is fast and, crucially, the production tag points at the byte-for-byte same image QA tested.

> **THIS IS THE WHOLE POINT OF THE PIPELINE**
> Lines 174–185 are where “build once, promote later” actually happens. The comment says it plainly: *No rebuilding*. Same image, same digest, new name.

Job 2 — update-prod-manifests `lines 204–315`

This job writes the new production tags into the production manifests on `main` and commits them — the production equivalent of `update-qa-manifests`.

```
201   # ============================================================
202   # 2. UPDATE PROD KUBERNETES MANIFESTS
203   # ============================================================
204   update-prod-manifests:
205     name: Update Production Manifests
206
207     needs:
208       - promote-images-to-prod
209
210     runs-on: ubuntu-latest
211
212     permissions:
213       contents: write
```

Lines 207–208 `needs: promote-images-to-prod` — only once the production tags exist in ECR. Writing a tag into a manifest before the image exists would make any deploy of that manifest fail with an image pull error.

Lines 212–213 `contents: write` so the bot can push to `main`. No AWS access is needed in this job.

```
217   # ------------------------------------------------------------
218   # Checkout MAIN branch
219   # ------------------------------------------------------------
220   - name: Checkout main branch
221     uses: actions/checkout@v7
222     with:
223       ref: main
224       fetch-depth: 0
225       token: ${{ secrets.GITHUB_TOKEN }}
```

Line 223 `ref: main` — check out the current tip of `main`, where the production manifests live.

Line 224 `fetch-depth: 0` — full history, so `git log` in the next step works normally.

Line 225 The token used for the later `git push`.

```
228   # ------------------------------------------------------------
229   # Verify production manifests
230   # ------------------------------------------------------------
231   - name: Verify production manifests
232     shell: bash
233     run: |
234       set -euo pipefail
235
236       echo "Branch:"
237       git branch --show-current
238
239       echo
240       echo "Commit:"
241       git log -1 --oneline
242
243       echo
244       echo "Production manifests:"
245       ls -la k8-manifests/prod/
246
247       test -f k8-manifests/prod/backend.yaml
248       test -f k8-manifests/prod/frontend.yaml
249
250       echo "backend.yaml found"
251       echo "frontend.yaml found"
```

Lines 236–237 `git branch --show-current` prints the checked-out branch — it should say `main`.

Lines 239–241 `git log -1 --oneline` prints the latest commit, so the log records exactly which state of `main` we are about to modify. `echo` with no arguments prints a blank line for readability.

Lines 243–245 `ls -la` lists the production manifest folder, with sizes and timestamps.

Lines 247–251 Fail fast if either manifest is missing.

```
254   # ----------------------------------------------------------
255   # Replace image references with PROD image tags
256   # ----------------------------------------------------------
257   - name: Update backend and frontend production images
258     shell: bash
259
260     env:
261       BACKEND_PROD_IMAGE: ${{ needs.promote-images-to-prod.outputs.backend_prod_image }}
262       FRONTEND_PROD_IMAGE: ${{ needs.promote-images-to-prod.outputs.frontend_prod_image }}
263
264     run: |
265       set -euo pipefail
266
267       sed -Ei \
268         "s|^([[:space:]]*)image:.*|\1image: ${BACKEND_PROD_IMAGE}|" \
269         k8-manifests/prod/backend.yaml
270
271       sed -Ei \
272         "s|^([[:space:]]*)image:.*|\1image: ${FRONTEND_PROD_IMAGE}|" \
273         k8-manifests/prod/frontend.yaml
```

Lines 260–262 Read the production image names from the first job's outputs into environment variables.

Lines 267–273 The same `sed` substitution as in QA, applied to the production manifests: keep the indentation, replace whatever follows `image:` with the new production image. The same caution applies — every `image:` line in each file is replaced.

```
276   # ----------------------------------------------------------
277   # Verify updated image references
278   # ----------------------------------------------------------
279   - name: Show updated production images
280     shell: bash
281     run: |
282       set -euo pipefail
283
284       echo "Backend:"
285       grep -m1 "image:" k8-manifests/prod/backend.yaml
286
287       echo
288       echo "Frontend:"
289       grep -m1 "image:" k8-manifests/prod/frontend.yaml
```

Lines 284–289 Show the first `image:` line of each file so the log proves the change before it is committed.

```
292   # ------------------------------------------------------------
293   # Commit updated manifests to main
294   # ------------------------------------------------------------
295   - name: Commit updated production manifests
296     shell: bash
297     run: |
298       set -euo pipefail
299
300       git config user.name "github-actions[bot]"
301       git config user.email \
302         "41898282+github-actions[bot]@users.noreply.github.com"
303
304       git add \
305         k8-manifests/prod/backend.yaml \
306         k8-manifests/prod/frontend.yaml
307
308       if git diff --cached --quiet; then
309         echo "No manifest changes to commit"
310       else
311         git commit \
312           -m "Promote images to ${{ needs.promote-images-to-prod.outputs.prod_tag }} [skip ci]"
313
314         git push origin main
315       fi
```

Lines 300–302 Commit as the GitHub Actions bot, as in QA.

Lines 304–306 Stage the two production manifests.

Line 308 `git diff --cached --quiet` compares what is staged against the last commit. It exits with 0 when there is **no** difference and 1 when there is one. Used in an `if`, a 0 exit counts as true.

Line 309 So when nothing changed — for example, promoting the same images twice — we just log a message and skip the commit and push.

Lines 311–312 Otherwise commit with a message naming the production tag. `[skip ci]` stops the commit from starting any workflow that listens to pushes on `main`.

Line 314 Push to `main`. The push only happens when there was a commit to push — a cleaner pattern than the QA job's `|| echo`.

> **BRANCH PROTECTION ON MAIN**
> Most teams protect `main`. If it requires pull requests or status checks, this direct push by `github-actions[bot]` is rejected and the job fails. Either add GitHub Actions to the ruleset's bypass list or push with a dedicated GitHub App token that is allowed to bypass. Do not weaken the protection for everyone to make the bot work.

Job 3 — deploy-to-prod `lines 322–546`

The final job applies the production manifests to the `prod` namespace and confirms the rollout. Before touching anything, it verifies its identity, its namespace access and every permission it needs.

```
319   # ============================================================
320   # 3. DEPLOY TO PRODUCTION EKS
321   # ============================================================
322   deploy-to-prod:
323     name: Deploy to Production EKS
324
325     needs:
326       - update-prod-manifests
327
328     runs-on: ubuntu-latest
329
330     permissions:
331       contents: read
332       id-token: write
```

Lines 325–326 Runs only after the manifests on `main` were updated.

Lines 330–332 `contents: read` and `id-token: write` — read the code, log in to AWS.

```
336   # ------------------------------------------------------------
337   # Checkout latest MAIN
338   # ------------------------------------------------------------
339   - name: Checkout latest main branch
340     uses: actions/checkout@v7
341     with:
342       ref: main
343       fetch-depth: 0
344
345
346   # ------------------------------------------------------------
347   # Verify deployment manifests exist
348   # ------------------------------------------------------------
349   - name: Verify production manifest directory
350     shell: bash
351     run: |
352       set -euo pipefail
353
354       echo "Production manifests:"
355       ls -la k8-manifests/prod/
356
357       test -f k8-manifests/prod/backend.yaml
358       test -f k8-manifests/prod/frontend.yaml
359       test -f k8-manifests/prod/ingress.yaml
```

Lines 340–343 Check out `main` with full history. Because the previous job just pushed its commit, the tip of `main` now contains the new production tags — this is why we check out by branch name and not by the run's original SHA.

Lines 349–355 List the manifest folder in the log.

Lines 357–359 Require all three files, including `ingress.yaml`. A production deploy without its ingress would leave the application unreachable.

```
362   # ------------------------------------------------------------
363   # GitHub OIDC -> GitHubActionsEKSProdRole
364   # ------------------------------------------------------------
365   - name: Configure AWS credentials from GitHub OIDC
366     uses: aws-actions/configure-aws-credentials@v6.3.0
367     with:
368       role-to-assume: ${{ secrets.AWS_PROD_ROLE_TO_ASSUME }}
369       aws-region: ${{ env.AWS_REGION }}
370
371
372   # ------------------------------------------------------------
373   # Verify assumed IAM Role
374   # ------------------------------------------------------------
375   - name: Verify AWS identity
376     run: aws sts get-caller-identity
377
378
379   # ------------------------------------------------------------
380   # Install kubectl
381   # ------------------------------------------------------------
382   - name: Setup kubectl
383     uses: azure/setup-kubectl@v4
```

Lines 365–369 OIDC login with the production role. The comment names the actual role, `GitHubActionsEKSProdRole`.

Lines 375–376 Print the assumed identity.

Lines 382–383 Install `kubectl`.

```
386   # ------------------------------------------------------------
387   # Generate kubeconfig
388   #
389   # Requires:
390   # eks:DescribeCluster
391   # ------------------------------------------------------------
392   - name: Generate kubeconfig for EKS
393     shell: bash
394     run: |
395       set -euo pipefail
396
397       aws eks update-kubeconfig \
398         --region "$AWS_REGION" \
399         --name "$EKS_CLUSTER_NAME"
```

Lines 386–391 The comment records an important detail: generating a kubeconfig needs the IAM permission `eks:DescribeCluster` on the production role.

Lines 397–399 Write `~/.kube/config` for `my-cluster`.

Verifying access before deploying

```
402   # ------------------------------------------------------------
403   # Verify namespace access
404   #
405   # IMPORTANT:
406   # Do NOT use:
407   #
408   # kubectl cluster-info
409   # kubectl get nodes
410   #
411   # because this PROD role is intentionally scoped only
412   # to the prod namespace.
413   # ------------------------------------------------------------
414   - name: Verify PROD namespace access
415     shell: bash
416     run: |
417       set -euo pipefail
418
419       echo "Checking access to namespace: $K8S_NAMESPACE"
420
421       kubectl get pods \
422         -n "$K8S_NAMESPACE"
423
424       kubectl get services \
425         -n "$K8S_NAMESPACE"
```

Lines 403–413 This comment is worth reading carefully. The production role is mapped to Kubernetes permissions **only inside the `prod` namespace**. Cluster-wide commands such as `kubectl cluster-info` or `kubectl get nodes` would be refused with *Forbidden* — not because something is broken, but because the role is correctly restricted. So the check uses namespaced commands only.

Line 419 Log which namespace we are checking.

Lines 421–425 `kubectl get pods -n prod` and `kubectl get services -n prod`. If the role cannot read the namespace, or the namespace does not exist, the step fails here, before any change is made.

```
428   # ------------------------------------------------------------
429   # Verify required Kubernetes permissions
430   # ------------------------------------------------------------
431   - name: Verify Kubernetes deployment permissions
432     shell: bash
433     run: |
434       set -euo pipefail
435
436       check_permission() {
437
438         RESOURCE="$1"
439
440         RESULT=$(kubectl auth can-i create "$RESOURCE" \
441           -n "$K8S_NAMESPACE")
442
443         echo "$RESOURCE -> $RESULT"
444
445         if [ "$RESULT" != "yes" ]; then
446           echo "ERROR: Missing permission to create $RESOURCE"
447           exit 1
448         fi
449       }
450
451       check_permission deployments
452       check_permission services
453       check_permission configmaps
454       check_permission secrets
455       check_permission ingresses.networking.k8s.io
```

Line 436 Defines a small Bash function, `check_permission`, so we do not repeat the same five lines five times.

Line 438 `RESOURCE="$1"` — the first argument passed to the function.

Lines 440–441 Ask the cluster “can I create this resource in `prod`?” and capture the answer (`yes` or `no`) in `RESULT`.

Line 443 Log it, for example `deployments -> yes`.

Lines 445–448 If the answer is anything other than `yes`, print which permission is missing and stop the job.

Lines 451–455 Check the five resource types the production manifests may contain: Deployments, Services, ConfigMaps, Secrets and Ingresses. Production checks more than QA because the whole `prod/` folder is applied in the next steps, and it may hold configuration and secret objects as well.

A SMALL DETAIL IN THE ERROR PATH

When `kubectl auth can-i` answers `no`, it also exits with code 1. Under `set -e`, a failing command inside `RESULT=$(...)` stops the script on the spot — so the job does fail correctly, but the friendly “ERROR: Missing permission…” message on line 446 never prints. To get the message, write the line as `RESULT=$(kubectl auth can-i create "$RESOURCE" -n "$K8S_NAMESPACE" || true)`.

Deploying and confirming the rollout

```
458   # ------------------------------------------------------------
459   # Show exact images before deployment
460   # ------------------------------------------------------------
461   - name: Show production images before deployment
462     shell: bash
463     run: |
464       set -euo pipefail
465
466       echo "Backend image:"
467       grep -m1 "image:" \
468         k8-manifests/prod/backend.yaml
469
470       echo
471       echo "Frontend image:"
472       grep -m1 "image:" \
473         k8-manifests/prod/frontend.yaml
474
475
476   # ------------------------------------------------------------
477   # Deploy PROD manifests
478   # ------------------------------------------------------------
479   - name: Deploy production manifests
480     shell: bash
481     run: |
482       set -euo pipefail
483
484       kubectl apply \
485         -f k8-manifests/prod/
```

Lines 466–473 Print the image each manifest is about to deploy — the final human-readable confirmation in the log before the cluster changes.

Lines 484–485 `kubectl apply -f k8-manifests/prod/` applies **every** manifest file in the folder in one command (`-f` with a directory reads all `.yaml`, `.yml` and `.json` files in it, not subfolders). QA applied three named files; production applies the folder, so any ConfigMap or Secret manifest placed there is deployed too.

```
488   # ------------------------------------------------------------
489   # Verify backend rollout
490   # ------------------------------------------------------------
491   - name: Verify backend rollout
492     shell: bash
493     run: |
494       set -euo pipefail
495
496       kubectl rollout status deployment/backend \
497         -n "$K8S_NAMESPACE" \
498         --timeout=300s
499
500
501   # ------------------------------------------------------------
502   # Verify frontend rollout
503   # ------------------------------------------------------------
504   - name: Verify frontend rollout
505     shell: bash
506     run: |
507       set -euo pipefail
508
509       kubectl rollout status deployment/frontend \
510         -n "$K8S_NAMESPACE" \
511         --timeout=300s
```

Lines 496–498 Wait up to five minutes for the backend Deployment's rollout to complete; fail the job if it does not.

Lines 509–511 The same for the frontend.

> **IF A PRODUCTION ROLLOUT TIMES OUT**
> The job fails, but Kubernetes keeps the old pods serving traffic until new ones are ready, so a failed rollout usually does not mean an outage. Inspect with `kubectl describe pod` and `kubectl logs` in `prod`. To roll back the GitOps way, set the `image:` lines in `k8-manifests/prod/` back to the previous `prod-…` tag (visible in the `main` history) and apply. `kubectl rollout undo` is faster in an emergency, but it leaves Git saying something different from the cluster, so follow it up with the manifest change.

```
514   # ------------------------------------------------------------
515   # Show final production resources
516   # ------------------------------------------------------------
517   - name: Show production resources
518     shell: bash
519     run: |
520       set -euo pipefail
521
522       echo "======================================"
523       echo "PODS"
524       echo "======================================"
525
526       kubectl get pods \
527         -n "$K8S_NAMESPACE" \
528         -o wide
529
530
531       echo
532       echo "======================================"
533       echo "SERVICES"
534       echo "======================================"
535
```

```
536       kubectl get services \
537         -n "$K8S_NAMESPACE"
538
539
540       echo
541       echo "====================================="
542       echo "INGRESS"
543       echo "====================================="
544
545       kubectl get ingress \
546         -n "$K8S_NAMESPACE"
```

Lines 522–546 Print the pods, services and ingress in `prod`, separated by banner lines so the three sections are easy to find in the log. This output is the final record of the release in the run log.

CHAPTER 5

A Release, End to End

Here is what one feature looks like as it travels through both pipelines — what each person does, and what changes in Git, ECR and the cluster at each point.

Monday — the change reaches QA

A developer's pull request with a backend fix is reviewed and merged into `qa`. The merge is a push to `qa` that touches `backend/**`, so **Enterprise DevSecOps QA CI/CD** starts on its own. Suppose it is run number 42, first attempt, and the merge commit is `a1b2c3d…`.

The six stage-1 jobs run side by side for a few minutes. Maven tests pass, the frontend builds, and the four scanners upload their reports. The `docker` job builds both images, scans them, writes the SBOMs, and pushes:

```
<account-id>.dkr.ecr.ap-south-1.amazonaws.com/projectops-backend:qa-42-1-a1b2c3d
<account-id>.dkr.ecr.ap-south-1.amazonaws.com/projectops-frontend:qa-42-1-a1b2c3d
```

`update-qa-manifests` writes those names into the QA manifests and pushes a bot commit to `qa`:

```
github-actions[bot]  Update QA ECR images to qa-42-1-a1b2c3d [skip ci]
```

`deploy-to-k8s` applies the manifests, waits for both rollouts and prints the ingress address. SonarQube finishes its analysis around the same time and posts the quality gate result on the server.

Monday to Wednesday — testing and review

The testing team works against the QA ingress. Meanwhile the security reviewer downloads the artifacts from run 42 — the Gitleaks, Semgrep, Checkov and Trivy reports — and checks the SonarQube dashboard. Because every scanner is in report mode, this review is where findings are actually acted on: a CRITICAL in the image report becomes a ticket, or blocks sign-off.

QA signs off. From this point the team stops merging into `qa` until the release is out, so that the tip of `qa` keeps pointing at the approved build.

Thursday — promotion to production

The release manager opens **Actions → Enterprise Production CD Pipeline → Run workflow**, keeps the branch as `main`, and presses the button. Suppose this is production run 17, and the tip of `main` is `9f8e7d6…`.

1. `promote-images-to-prod` assumes the production role, reads `qa-42-1-a1b2c3d` from both QA manifests, pulls those two images, tags them `prod-17-1-9f8e7d6` and pushes the new tags. The digest in ECR is the same for both tags.
2. `update-prod-manifests` writes the production names into `k8-manifests/prod/` on `main` and pushes `Promote images to prod-17-1-9f8e7d6 [skip ci]`.
3. `deploy-to-prod` checks out the new tip of `main`, assumes the production role again, verifies it can read `prod` and create all five resource types, applies the whole `prod/` folder and waits for both rollouts.

The release is done. Git now records the release on both branches, ECR holds both tags on the same image, and the run log shows the identity that deployed it and the pods that are running.

> **WHERE TO FIND WHAT AFTERWARDS**
> **What is in QA right now?** The `image:` lines in `k8-manifests/qa/` on `qa`. **What is in production?** The same lines in `k8-manifests/prod/` on `main`. **When did it change and who triggered it?** The bot commits on each branch, and the run they point to. **What was in the image?** The SBOM artifacts from the QA run that built it.

CHAPTER 6

Hardening Notes

The pipelines work as they are. These are the changes we make as a project matures — roughly in the order we would do them. None of them change the overall design.

1. Turn the scanners from reporting into gating

Today, every scanner records findings and moves on. The usual path is to start enforcing on the most serious, most actionable findings first, so the team is not blocked by noise:

- **Trivy (FS and image):** change `--exit-code 0` to `--exit-code 1`. With `--severity HIGH,CRITICAL --ignore-unfixed` already in place, the job only fails on serious findings that actually have a fix. Use a `.trivyignore` file for accepted risks, with a comment and an expiry date for each entry.
- **Gitleaks:** remove `--exit-code 0`. A committed secret should always stop the line — and should always be rotated, not just deleted.
- **Semgrep and Checkov:** remove `|| true` once the existing findings are fixed or baselined; for Checkov, `--skip-check` lists the deliberate exceptions.
- **SonarQube:** remove `continue-on-error: true` from the quality gate and add `sonarqube` to the `needs:` of the `docker` job, so a failed gate stops the image from being pushed.

Keep uploading the reports even when a scan fails: add `if: always()` to the upload steps, otherwise a failing scan skips the upload and you lose the very report that explains the failure.

2. Use GitHub Environments for production

Create a `production` environment in the repository settings and add `environment: production` to the production jobs. That gives us required reviewers (a second person must approve before the job starts), a wait timer if wanted, and environment-scoped secrets so `AWS_PROD_ROLE_TO_ASSUME` is only readable by jobs that target production. The OIDC `sub` claim then becomes `repo:<OWNER>/<REPO>:environment:production`, and the production role's trust policy should be updated to match.

3. Make tags immutable in ECR

Turn on **tag immutability** for both repositories. Our tags already include the run number and attempt, so we never intend to overwrite one; immutability turns that intention into a guarantee. Nobody can push a different image under an existing `prod-…` tag, by accident or otherwise.

4. Pin what we depend on

`zricethezav/gitleaks:latest`, Trivy `version: latest` and `anchore/sbom-action@v0` can change under us between two runs. Pin container images and tools to explicit versions, and pin third-party actions to a full

commit SHA (with the version in a comment). Dependabot can keep the pins up to date with reviewed pull requests.

5. Promote inside the registry

Pulling two images onto the runner just to push a new tag works, but costs time and bandwidth for large images. The same result can be achieved on the registry side, without downloading layers — for example with `crane tag`, or with the AWS CLI by reading the image manifest (`aws ecr batch-get-image`) and writing it back under the new tag (`aws ecr put-image`). The digest stays identical either way.

6. Smaller tidy-ups

- **Full history for Gitleaks:** add `fetch-depth: 0` to its checkout so every commit in a push is scanned.
- **Targeted image updates:** replace the `sed` rewrites with `yq` or `kubectl set image` on a named container, so adding a sidecar never breaks a deploy.
- **Avoid the double Maven build:** upload `backend/target/classes` as an artifact from `backend-test` and download it in `sonarqube`, instead of running `mvn verify` twice. Uploading the Surefire and JaCoCo reports too would give SonarQube real test coverage numbers.
- **Clearer permission errors in production:** add `|| true` inside the `kubectl auth can-i` capture so the friendly error message is printed (see Job 3 of Chapter 4).
- **Optional pinned promotion:** add a `workflow_dispatch` input for an explicit QA tag, defaulting to "read from the QA manifest", for the rare case where you need to promote a specific earlier build.
- **Post-deploy smoke test:** after the rollout, `curl` a health endpoint through the ingress and fail the job if it does not answer.

CHAPTER 7

Quick Reference

Triggers and safety settings

	QA pipeline	Production pipeline
Starts on	Push to `qa` touching the listed paths, or Run workflow	Run workflow only
Concurrency	`qa-<ref>`, cancels older runs	`production`, queues — never cancels
AWS role	`AWS_QA_ROLE_TO_ASSUME`	`AWS_PROD_ROLE_TO_ASSUME` (prod namespace only)
Builds images?	Yes — backend and frontend	No — retags the QA images
Image tag	`qa-<run>-<attempt>-<sha7>`	`prod-<run>-<attempt>-<sha7>`
Manifests updated	`k8-manifests/qa/` on `qa`	`k8-manifests/prod/` on `main`
Applies	backend, frontend, ingress (three files)	the whole `k8-manifests/prod/` folder
Namespace	`qa`	`prod`

Artifacts produced by every QA run

Artifact	From job	Contents
`gitleaks-report`	gitleaks-scan	Redacted secret findings (JSON)
`semgrep-report`	semgrep-scan	SAST findings for backend and frontend (JSON)
`checkov-k8s-report`	checkov-k8s	Kubernetes manifest checks (JSON)
`checkov-dockerfile-report`	checkov-dockerfile	Dockerfile checks (JSON)
`backend-trivy-fs-report`	backend-test	Maven dependency CVEs, HIGH/CRITICAL with fixes
`frontend-trivy-fs-report`	frontend-build	npm dependency CVEs, HIGH/CRITICAL with fixes
`trivy-image-reports`	docker	OS and library CVEs in both images
SBOM artifacts	docker	SPDX JSON inventory of each image (named by the SBOM action)

Commands worth remembering

```
# Which images are QA and production running, according to Git?
git show qa:k8-manifests/qa/backend.yaml   | grep -m1 image:
git show main:k8-manifests/prod/backend.yaml | grep -m1 image:

# Which images is the cluster actually running?
kubectl get deploy -n prod -o wide        # IMAGES column shows the running tags

# Do two tags point at the same image? (same digest = same image)
aws ecr describe-images --repository-name projectops-backend \
  --image-ids imageTag=qa-42-1-a1b2c3d imageTag=prod-17-1-9f8e7d6 \
  --query 'imageDetails[].[imageTags,imageDigest]'

# What can the current identity do in prod?
kubectl auth can-i --list -n prod
```

APPENDIX

If You Are Teaching This

A sequence that works well in a live session or video:

1. Start with Figure 1 and the build-once idea before showing any YAML. Ask: “why not just rebuild for production?”
2. Walk the QA header, then show Figure 2 and have the audience derive it from the `needs:` lines themselves.
3. Go through one scanner job in full detail (Gitleaks is shortest), then cover the other three by their differences.
4. Spend the most time on the `docker` job: OIDC, `GITHUB_ENV` versus `GITHUB_OUTPUT`, `load: true` before scanning, push last.
5. Run a real QA pipeline, open the artifacts and the bot commit on `qa`.
6. Run the production pipeline and show the identical digests for the QA and production tags in ECR.
7. Close with Chapter 6 — it is where the interview questions are.